

Final Year Project Report

RISC-V SCALAR CRYPTO EXTENSION

B.S. in Computer Engineering, Batch 2020

Project Advisor

Saqib Hussain

Assistant Professor

SSUET



Submitted By:

Ayesha Khan (GL)	2020-CE-132
Neha Shakil	2020-CE-152
M. Ali Panhwar	2020-CE-130

DEPARTMENT OF COMPUTER ENGINEERING

Sir Syed University of Engineering & Technology

University Road Karachi - 75300

February 2024

Preface

This project represents the culmination of our journey through the development and integration of a RISC-V Scalar Crypto Extension, a project that aims to fortify the security capabilities of the RISC-V architecture. At the heart of our endeavor lies the ESP32C3 microcontroller, a cornerstone for efficient hardware execution, around which we've built a system capable of advanced cryptographic operations, specifically AES encryption and decryption with a 128-bit key. The creation of a user-friendly interface, encompassing an LCD module and keyboard, allows for immediate interaction and secure data processing, marking a significant milestone in our project.

On the software spectrum, we've dedicated our efforts to constructing a website that not only enhances user experience but also ensures seamless encryption and decryption functionalities, all while adhering to the stringent AES standards. The integration of the GNU Toolchain compiler has been pivotal in achieving a transition of RISC-V code from software to hardware, embodying our vision of a comprehensive solution for secure data processing. Throughout this project, we've navigated through a series of challenges, from adhering to cryptographic standards to ensuring compatibility across various platforms. Our assumptions were tested against the practicality of integrating the ESP32C3 and the availability of essential development tools, pushing us to adapt and innovate continually.

This preface serves not only as an introduction to the technicalities and achievements of our project but also as a reflection of our journey. The process of developing the RISC-V Scalar Crypto Extension has been both demanding and rewarding, offering us invaluable insights into the realm of secure data processing within the RISC-V architecture. We aspire that our project not only bridges the current gaps in the field but also significantly contributes to the advancement of secure applications within RISC-V systems, marking a pivotal step towards enhanced cryptographic security.

Acknowledgements

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of this research. First and foremost, I am immensely thankful to Almighty Allah for granting me strength, wisdom, and perseverance throughout this journey. My deepest appreciation goes to my main supervisor, Assistant Professor Saqib Hussain Siddiqui, whose guidance, expertise, and unwavering support have been invaluable. I am also grateful to my Co-supervisor, Professor Dr. Abid Muhammad Khan, for their insightful feedback and constructive criticism that significantly enriched the quality of this research. I extend my thanks to the dedicated academic staff in my department for their encouragement and willingness to share their knowledge. Special appreciation is reserved for the technical and support staff whose assistance was crucial in conducting experiments and gathering data. I want to acknowledge the valuable contributions of academic staff from other departments who generously shared their insights, enhancing the interdisciplinary nature of this research.

Additionally, my gratitude extends to (MERL) Usman Univeristy for their collaboration and resources that enhanced the scope of this study. My appreciation also goes to my team members, family, and friends for their unwavering support, understanding, and encouragement throughout this challenging endeavor. In conclusion, I recognize and thank all those who have played a role in shaping this research. Your collective efforts have left an indelible mark on this work, and for that, I am truly grateful.

Abstract

This project focuses on addressing the security vulnerabilities within the RISC-V architecture through the development and integration of a RISC-V Scalar Crypto Extension. Recognizing the critical need for enhanced cryptographic measures, this study introduces a scalable and standardized solution to fill the existing research voids. By employing the ESP32C3 microcontroller, we ensure efficient hardware execution, while comprehensive planning addresses both the hardware and software facets of the project. A pivotal aspect of our work is the successful implementation of AES encryption and decryption with a 128-bit key, which significantly mitigates a notable gap in current literature. Furthermore, the project facilitates real-time secure data processing through a system interface, including an LCD module and keyboard, and the development of a user-friendly website. These elements, combined with the integration of the GNU Toolchain compiler, enable a seamless transition from software to hardware. Our findings, validated by clock diagrams, confirm the efficacy of the crypto extension. The research not only enhances the RISC-V architecture's security capabilities but also offers practical insights and solutions for secure data processing, marking a substantial contribution to the domain of cryptographic security. Through the development of the RISC-V Scalar Crypto Extension and its integration into the broader ecosystem, this study embodies a holistic approach to bolstering security measures, providing significant advancements in the field of secure data processing on RISC-V platforms.

Chapter 1

Introduction

1.1 Overview

In today's digital age, the security of sensitive data is paramount amidst the increasing prevalence of cyber threats and data breaches. Leveraging the ESP32C3 microcontroller, this project aims to develop an efficient and secure data encryption and decryption system to address this pressing need [1]. Key deliverables include the development and integration of the AES-128 bit Algorithm into an API, as well as the creation of a user-friendly website. By utilizing industry-standard encryption algorithms, the project seeks to create a holistic solution for safeguarding sensitive information from unauthorized access. With a focus on usability and accessibility, the project aligns with current trends in cybersecurity, aiming to provide a practical and effective means of protecting data in digital environments. Ultimately, by leveraging cutting-edge technology and robust encryption measures, the project endeavors to contribute to the broader cybersecurity landscape and mitigate the risks posed by cyber threats and data breaches.

1.2 Objectives

The objectives are the following:

- Discover and prove the feasibility and effectiveness of leveraging the ESP32C3 microcontroller for secure data encryption and decryption, culminating in the development and integration of the AES-128 bit Algorithm into an API [2].
- Address the problem of data security in digital systems by providing a practical solution that utilizes industry-standard encryption algorithms, such as AES-128, and includes the creation of a user-friendly website.
- Work on this problem to contribute to enhancing cybersecurity measures and protecting sensitive information from unauthorized access through the implementation of robust encryption and decryption systems, including the development of the AES-128 bit Algorithm API [3].
- Respond to the advisor's assignment to foster learning in the areas of embedded systems, cryptography, and cybersecurity, aiming to develop practical skills in designing secure systems, which will be demonstrated through the completion of the project's key deliverables.
- Ensure that readers understand the significance of securing sensitive data in digital systems and appreciate the practical implementation of encryption and decryption techniques using microcontroller-based systems, with a particular emphasis on the development and integration of the AES-128 bit Algorithm into an API and the creation of a user-friendly website.

1.2.1 System Features

The system features are following:

- The project focuses on evaluating the ESP32C3 microcontroller's capabilities in secure data encryption/decryption and real-time AES encryption, alongside as-

sessing the usability of the LCD module/keyboard. Additionally, it involves validating a web app for seamless file encryption/decryption and verifying GNU Toolchain compiler integration for RISC-V code execution. These evaluations provide valuable insights into the microcontroller's role in enhancing embedded systems and cybersecurity. Key deliverables include developing and integrating the AES-128 bit Algorithm into an API, creating a user-friendly website, and ensuring seamless encryption/decryption functionality in the web application.

- Amid rising cyber threats, there's an urgent need for robust data security. Advancements in microcontroller technology offer a chance to develop efficient encryption systems. This project aims to contribute to cybersecurity by addressing this need. It seeks to enhance digital infrastructure resilience against emerging threats, improving security for organizations and individuals.
- The advisor likely assigned this problem to challenge the student to apply their knowledge of microcontrollers, cryptography, and software development in a real-world context. This project offers hands-on experience in designing secure systems, enhancing practical skills and problem-solving abilities in cybersecurity. It provides valuable insights applicable to addressing pressing data security concerns in digital environments.
- This project underscores the critical importance of data security and the consequences of insufficient protection measures. By exploring microcontroller capabilities, especially the ESP32C3, readers gain insights into implementing secure encryption systems, deepening their understanding of cybersecurity. The project also addresses the complexities of integrating hardware and software for holistic security solutions, emphasizing the significance of adhering to industry-standard cryptographic algorithms like AES. Additionally, practical demonstrations of the GNU Toolchain compiler showcase its role in enhancing embedded system security.

1.3 Summary

In summary, this chapter outlines the critical need for robust data security solutions, leveraging the ESP32C3 microcontroller. Objectives include proving the feasibility of microcontroller-based encryption, fostering practical skills in secure system design, and contributing to cybersecurity measures. Key deliverables involve developing an AES-128 bit Algorithm API, a user-friendly website, and ensuring seamless encryption/decryption functionality. System requirements include assessing microcontroller capabilities for secure data encryption/decryption and validating a web app for file encryption/decryption. Through these efforts, the project aims to enhance data security and mitigate cyber threats, contributing to cybersecurity advancements.

Chapter 2

Literature Review

1. *Efficient Cryptography on 32-bit RISC-V Processors*

This research paper explores the potential for enhancing cryptographic primitives using the RISC-V instruction set architecture (ISA). [4] The authors demonstrate the feasibility of enhancing cryptographic performance on RISC-V platforms by providing optimized assembly versions of ChaCha, bit-sliced AES, table-based AES, and the Keccak-f[1600] permutation for the RV32I instruction set. The paper evaluates the efficacy of arbitrary precision integer arithmetic in public key cryptography and forecasts potential benefits from incorporating RISC-V enhancements. This work provides valuable insights for enhancing cryptographic implementations on the RISC-V platform, shaping future design decisions for RISC-V extensions and implementations [5].

2. *The design of scalar AES Instruction Set Extensions for RISC-V*

This paper discusses the need of efficiently running AES on computing platforms that do not have standardized Instruction Set Extensions (ISEs), such as RISC-V [6]. The authors conduct a study of the latest ISEs for AES and test five distinct extensions, one of which is a new technique. Their research shows notable enhancements in the performance of AES-128 block encryption, suggesting the use of distinct ISEs for 32-bit and 64-bit base architectures [7]. The study

investigates using RISC-V's planned bit-manipulation extension to create an efficient AES-GCM implementation, which contributes to the current standardization work in RISC-V cryptography enhancements [8].

3. *Symmetric Cryptography on RISC-V: Performance Evaluation of Standardized Algorithms*

This paper discusses the urgent requirement for effective execution of cryptographic algorithms on the RISC-V architecture [9]. The study compares software solutions using basic instruction sets to those using specialized cryptographic instructions and hardware to analyze the performance and cost implications of each approach. The authors show substantial speed enhancements and memory conservation by utilizing cryptographic instruction set extensions in hand-optimized assembly language implementations of eleven cryptographic algorithms. This work aims to improve the effectiveness of cryptography implementations on RISC-V processors by introducing a new instruction to enhance algorithm efficiency [10].

4. *An energy-efficient crypto-extension design for RISC-V*

This study introduces a new method to improve cryptographic functions on RISC-V processors with a focus on reducing energy usage [11]. It is most likely examines the cryptographic extensions that are already in place for other architectures, assesses the energy-efficient cryptographic algorithms, and suggests optimized hardware designs for RISC-V, all of which contribute to the development of computing systems that are both secure and efficient in terms of power consumption [12].

2.1 Research Gap Analysis

1. *Efficient Cryptography on 32-bit RISC-V Processors*

Current research focuses on improving cryptographic primitives and suggesting

instruction set extensions for RISC-V processors, however there is a lack of attention given to the unique challenges and possibilities associated with 32-bit RISC-V processors. Additional study is required to explore effective cryptographic implementations designed for 32-bit RISC-V architectures, taking into account limitations such as restricted resources and power usage.

2. The design of scalar AES Instruction Set Extensions for RISC-V

While the literature provides insights into designing scalar AES instruction set extensions for RISC-V, there is a gap in addressing the scalability and standardization aspects across different RISC-V architectures. Future research could focus on developing standardized AES instruction set extensions that can seamlessly scale across various RISC-V platforms, ensuring compatibility and interoperability.

3. Symmetric Cryptography on RISC-V: Performance Evaluation of Standardized Algorithms

While previous studies evaluate the performance of standardized algorithms on RISC-V processors, there is a gap in comprehensive performance evaluations encompassing a wide range of symmetric cryptography algorithms beyond AES. Further research is needed to assess the efficiency of various cryptographic algorithms, including ChaCha, Keccak, and others, on RISC-V platforms to provide a holistic understanding of their suitability and performance characteristics.

4. An energy-efficient crypto-extension design for RISC-V:

While existing research proposes energy-efficient cryptographic extension designs for RISC-V processors, there is a gap in addressing the trade-offs between energy efficiency, performance, and security. Future research could focus on optimizing cryptographic extensions to strike a balance between energy consumption and cryptographic performance, considering real-world deployment scenarios and security requirements.

2.1.1 Bridging the Gap

1. *Development of a RISC-V Scalar Crypto Extension*

While existing research focuses on optimizing cryptographic primitives and proposing instruction set extensions for RISC-V processors, our project directly addresses this gap by developing and integrating a RISC-V Scalar Crypto Extension [7]. This extension enhances the security capabilities of the RISC-V architecture, providing a standardized and scalable solution for implementing advanced cryptographic functionalities.

2. *Efficient Hardware Execution with ESP32C3 Microcontroller*

Leveraging the ESP32C3 microcontroller for hardware execution, our project addresses the need for efficient cryptographic implementations tailored for 32-bit RISC-V architectures. By optimizing hardware execution, our project overcomes challenges related to limited resources and power consumption.

3. *Integration of Advanced Cryptographic Functionalities*

This project focuses on implementing advanced cryptographic functionalities, primarily AES encryption and decryption with a 128-bit key. This addresses the gap in the literature by providing practical solutions for securing data processing on RISC-V platforms, ensuring compatibility and interoperability across various RISC-V architectures.

4. *User-Friendly Interface and Seamless Encryption/Decryption*

The inclusion of an LCD module and keyboard for real-time interaction, along with a user-friendly website for seamless encryption and decryption, fills the gap in the literature by providing accessible and intuitive interfaces for secure data processing. This enhances the usability and practicality of cryptographic implementations on RISC-V platforms.

2.2 Summary

The literature review highlights gaps in optimizing cryptographic primitives for 32-bit RISC-V processors, standardizing AES instruction set extensions, evaluating a wide range of symmetric cryptography algorithms, and designing energy-efficient cryptographic extensions. Our project addresses these gaps by developing a RISC-V Scalar Crypto Extension tailored for 32-bit processors like ESP32C3, ensuring compatibility and scalability across various architectures. Leveraging efficient hardware execution with ESP32C3, we integrate advanced cryptographic functionalities, primarily AES encryption and decryption, while providing user-friendly interfaces for seamless interaction. By adhering to cryptographic standards and comprehensive planning, our project offers practical solutions for secure data processing, bridging the gap between theoretical research and real-world implementation on RISC-V platform.

Chapter 3

Problem Statement

3.1 Introduction

The existing issue is the limited security capabilities of the RISC-V architecture, requiring the development of an advanced RISC-V Scalar Crypto Extension. The deficiency lies in the absence of efficient hardware execution and seamless software-to-hardware integration for cryptographic functionalities, particularly AES encryption and decryption with a 128-bit key. The problem encompasses the lack of a user-friendly interface for real-time secure data processing and the need for standardized cryptographic standards adherence. This research aims to address these challenges by developing and integrating a comprehensive solution leveraging the ESP32C3 microcontroller, providing insights into the design, implementation, and integration of cryptographic functionalities within the RISC-V architecture.

3.2 Summary

Insufficient security capabilities in the RISC-V architecture, marked by inefficient hardware execution and incomplete software-to-hardware integration, prompt the need for an advanced RISC-V Scalar Crypto Extension. This research aims to address these deficiencies, offering a comprehensive solution leveraging the ESP32C3 microcontroller for optimized security functionalities.

Chapter 4

Technical Documents

4.1 DFD Diagrams

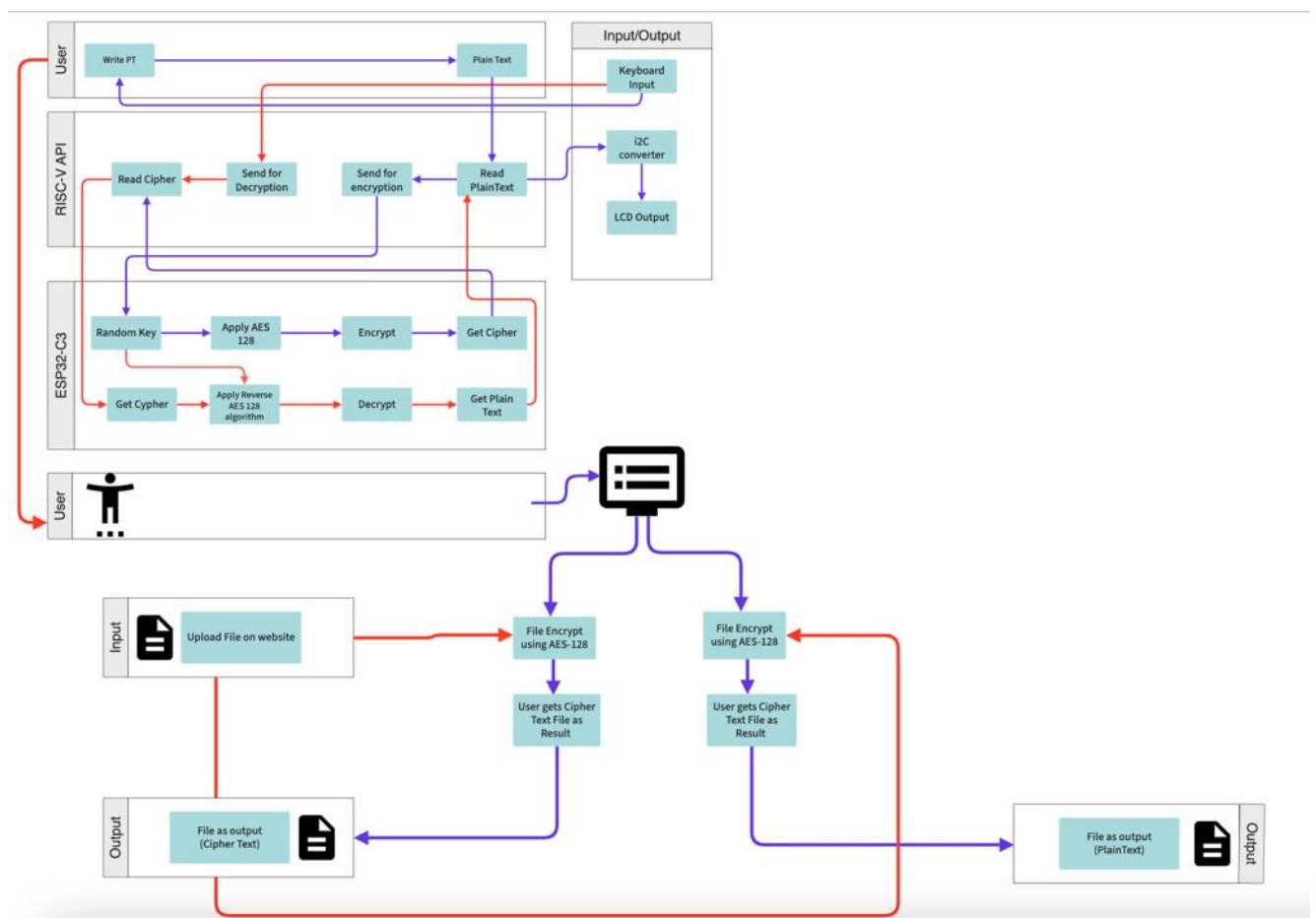


Figure 4.1: Line Box Diagram of Hardware and Software System

The project leverages the ESP32C3 microcontroller for efficient hardware execution. Users will interact with the system using a LCD module and keyboard, enabling real-time AES encryption and decryption with a 128-bit key. The software component features a user-friendly website designed for the seamless encryption of plain text files and decryption of cipher text files, utilizing the same AES standard [13]. A key aspect of the project is the integration of the GNU Toolchain compiler, ensuring a smooth flow of RISC-V code from the software to the ESP32C3 hardware. This streamlined operation is visually represented in Figure 4.1. This integration provides a holistic and secure solution for processing sensitive data.

4.2 Stakeholder Register

Table 4.1: Stakeholders Register

Stakeholder Name	Role	Expectation
Prof. Saqib Hussain	Supervisor	Guidance
Prof. M. Abid	Co-Supervisor	Guidance
Ayesha Khan	Student	Work
Neha Shakil	Student	Work
M.Ali Panhwar	Student	Work

The stakeholder register for the final year engineering project includes key individuals essential for its success. Sir Saqib, the supervisor, is responsible for providing overarching guidance and ensuring the project aligns with academic standards. Working alongside him is Sir Abid, the co-supervisor, who offers additional expertise and support. Ayesha Khan, M. Ali Panhwar, and Neha Shakil, the student team members, are tasked with actively participating in project tasks, demonstrating commitment, and collaborating effectively. Their dedication and contributions are crucial for achieving project milestones and delivering quality outcomes. Each stakeholder's role and expectations are clearly defined, fostering a collaborative and goal-oriented project environment.

4.3 Use Case Diagrams

In this encryption/decryption website, the user can upload a plain text file. The server will then encrypt it using the AES 128-bit algorithm and generate the corresponding cipher text file. The user can later decrypt this cipher text file using the AES 128-bit algorithm on the server, resulting in the regeneration of their original plain text file. This process is represented in Figure 4.2.

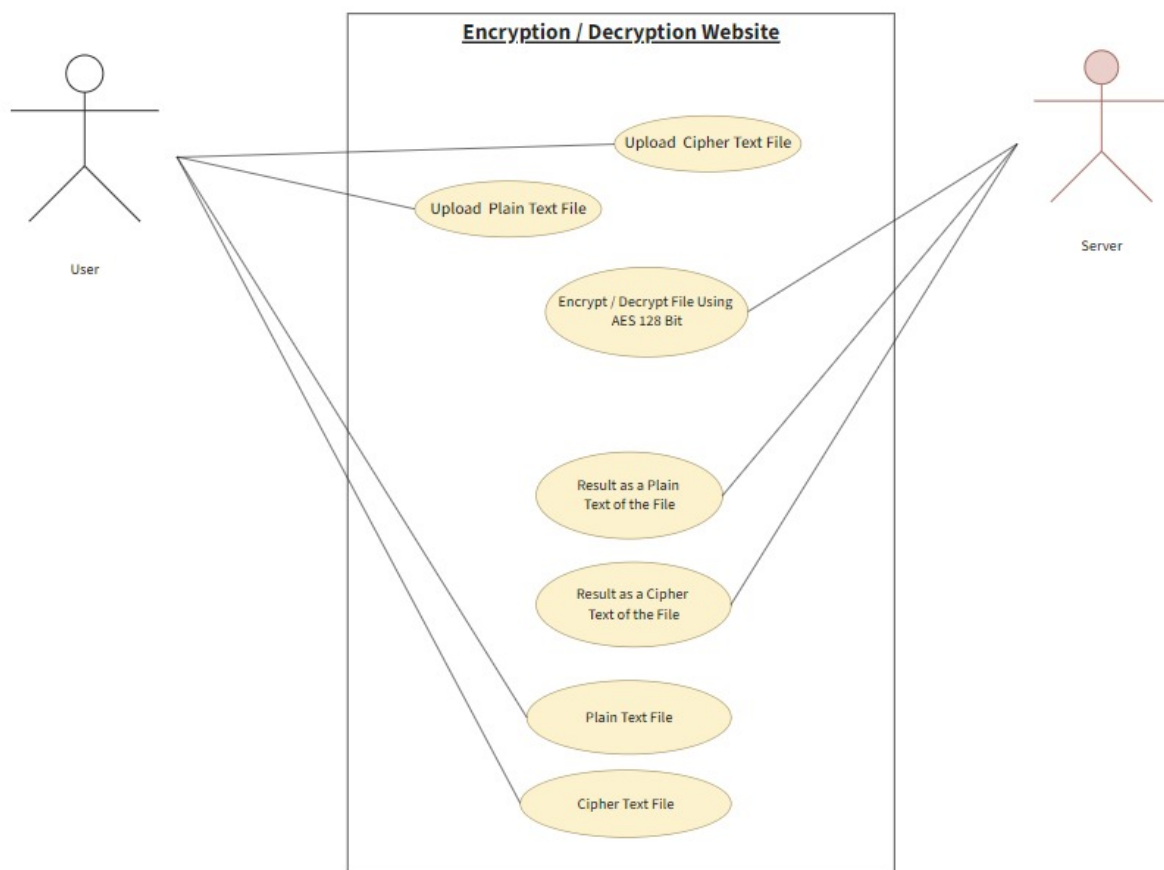


Figure 4.2: Use Case Of Encryption/Decryption Website

The user initiates the process by inputting plaintext through the keyboard. The RISC-V API process then takes charge of processing the user's input, preparing it for both encryption and decryption operations. The processed data is then handed over to the AES-128 encryption module, which performs the encryption operation. Subsequently, the encrypted data undergoes I2C conversion, adapting it for communication via the I2C protocol [14]. Simultaneously, the decrypted data follows a similar path, going through the AES-128 decryption process and I2C conversion. Finally, both

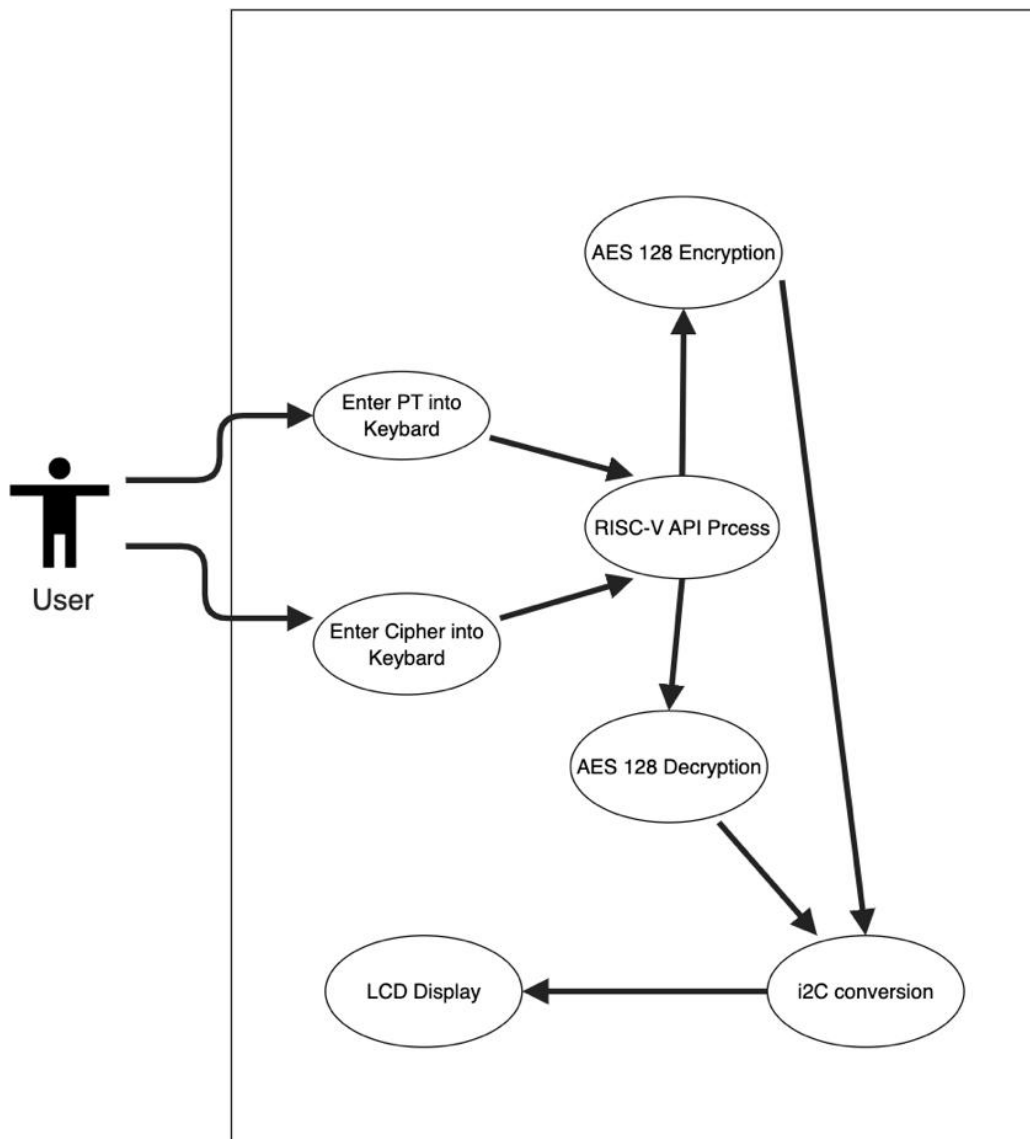


Figure 4.3: Use Case of Encryption and Decryption Hardware

the encrypted and decrypted data are transmitted to the LCD module, where they are displayed, providing real-time feedback to the user. This process is represented in Figure 4.3

4.4 Risk Register

Table 4.2: Risk Register

Risk Type	Likelihood	Impact	Risk Level	Mitigation Notes
Difficulty in sourcing required components	low	low	low	1. Start sourcing materials early. 2. Identify alternative suppliers. 3. Keep open communication with suppliers.
Equipment failure during critical stages	low	low	low	1. Regular maintenance of equipment. 2. Keep backup equipment or alternative methods available. 3. Test equipment beforehand.
Technical Challenges beyond team's expertise.	low	low	low	1. Conduct thorough research and feasibility studies. 2. Seek advice from mentors and faculty. 3. Allocate extra time for learning and experimentation.
Team Member conflicts	low	low	low	1. Foster a supportive team environment. 2. Have clear communication channels. 3. Develop contingency plans for redistributing workload.
Budget Constraints	low	low	low	1. Conduct thorough budget planning and estimation. 2. Look for potential sources of funding or sponsorship. 3. Prioritize spending based on project needs.
Scope creep leading to project delays	Moderate	High	High	1. Clearly define project scope and objectives. 2. Implement change control procedures. 3. Regularly review project progress against the plan.

Chapter 5

Methodology

5.1 Hardware Section

5.1.1 Hardware Selection:

1. Microcontroller:

- **Clock speed:** Higher clock speeds offer faster processing but may consume more power. Analyze the computational complexity of AES encryption and decryption to determine the necessary speed.
- **Character size and resolution:** 16x2 is suitable for basic text display.
- **Backlight:** Necessary for viewing in low-light conditions.
- **I2C interface:** Simplifies wiring and communication.

2. LCD Display:

- **Clock speed:** Higher clock speeds offer faster processing but may consume more power. Analyze the computational complexity of AES encryption and decryption to determine the necessary speed.
- **Available RAM and Flash memory:** Ensured sufficient RAM for code execution and data storage during encryption/decryption. Flash memory

stores the code and libraries.

- **Number of I/O pins:** Verified enough pins for LCD, keypad, and any additional sensors or communication modules.
- **Cost:** Balanced features and budget constraints.
- Considered ESP32-C3 DevModule for a budget-friendly option.
- Considered 16x2 I2C backpack.

3. Keypad:

- **Number of keys:** 4x4 offers basic input, while larger keypads provide more options.
- **I2C interface:** Simplifies wiring.
- **Additional functions:** Dedicated keys for specific actions (e.g., Enter, Clear).
- Considered 4x4 matrix keypads with I2C module.

5.1.2 Methodology Of Hardware:

1. Hardware Configuration and Connections:

Set up the ESP32-C3 microcontroller, I2C module, and keyboard according to their respective pin configurations and electrical specifications. Ensure proper power supply and grounding for all components.

2. Library Integration:

Include the base64 encoding and decoding libraries (base64.cpp, xbase64.cpp, and xbase64.h) in the project directory. Verify that the libraries are properly linked and accessible within the project environment.

3. I2C Communication Initialization:

Initialize the I2C communication interface on the ESP32-C3 microcontroller. Configure the I2C module with appropriate settings such as clock frequency and de-

vice addressing.

4. Keyboard Input Handling:

Implement code to read input from the connected keyboard via the I2C module. Utilize I2C communication protocols to receive keystroke data from the keyboard.

5. Base64 Encoding and Decoding Invocation:

Invoke the appropriate functions from the base64 libraries to encode or decode the received keyboard input. Pass the input data and its length to the respective encoding or decoding functions.

6. Data Processing:

Receive the encoded or decoded data from the base64 functions. Process the data as required for the application logic or functionality.

7. Error Handling:

Implement mechanisms to handle errors or exceptions that may occur during the encoding and decoding process. Include error-checking routines to ensure data integrity and robustness of the system.

8. Testing and Validation:

Test the functionality of the base64 encoding and decoding operations with various keyboard input scenarios. Validate the accuracy and reliability of the encoding and decoding processes.

9. Performance Optimization:

Optimize the code for performance and memory usage, if necessary. Identify and address any bottlenecks or inefficiencies in the encoding and decoding algorithms.

10. Documentation and Reporting:

Document the methodology used for integrating and utilizing the base64 libraries with the ESP32-C3, I2C module, and keyboard. Provide detailed documentation on library usage, function descriptions, and any specific configurations required for the hardware setup.

11. Deployment and Integration:

Deploy the finalized code to the ESP32-C3 microcontroller for real-world usage. Integrate the system with other components or modules as needed for the complete application solution. You can insert this methodology section into your document to provide a structured approach for integrating base64 encoding and decoding with the ESP32-C3, I2C module, and keyboard. Adjustments can be made based on specific project requirements and constraints.

5.1.3 Hardware Working

The system initiates the process through user input via the keyboard, marking the starting point of the operational flow. This input triggers a bifurcation into two primary pathways: Encryption and Decryption.

1. **For Encryption:** Upon selecting this option, the system advances to the encryption stage, wherein the provided data undergoes encryption leveraging the AES (128-bit) algorithm. Following this, the encrypted data status is conveyed to the user through the LCD display module, utilizing an I2C interface for communication. This feedback marks the completion of the encryption cycle, after which the system reverts to the initial stage, awaiting further input from the user via the keyboard.
2. **For Decryption:** Should the user opt for decryption, the system shifts to the decryption phase, applying the AES (128-bit) algorithm to decrypt the previously

encrypted data. Similar to the encryption path, the outcome of the decryption process is then relayed to the user via the LCD display module, also connected through an I2C interface. Post-display, the decryption process concludes, and the system resets to the start, ready for new user input.

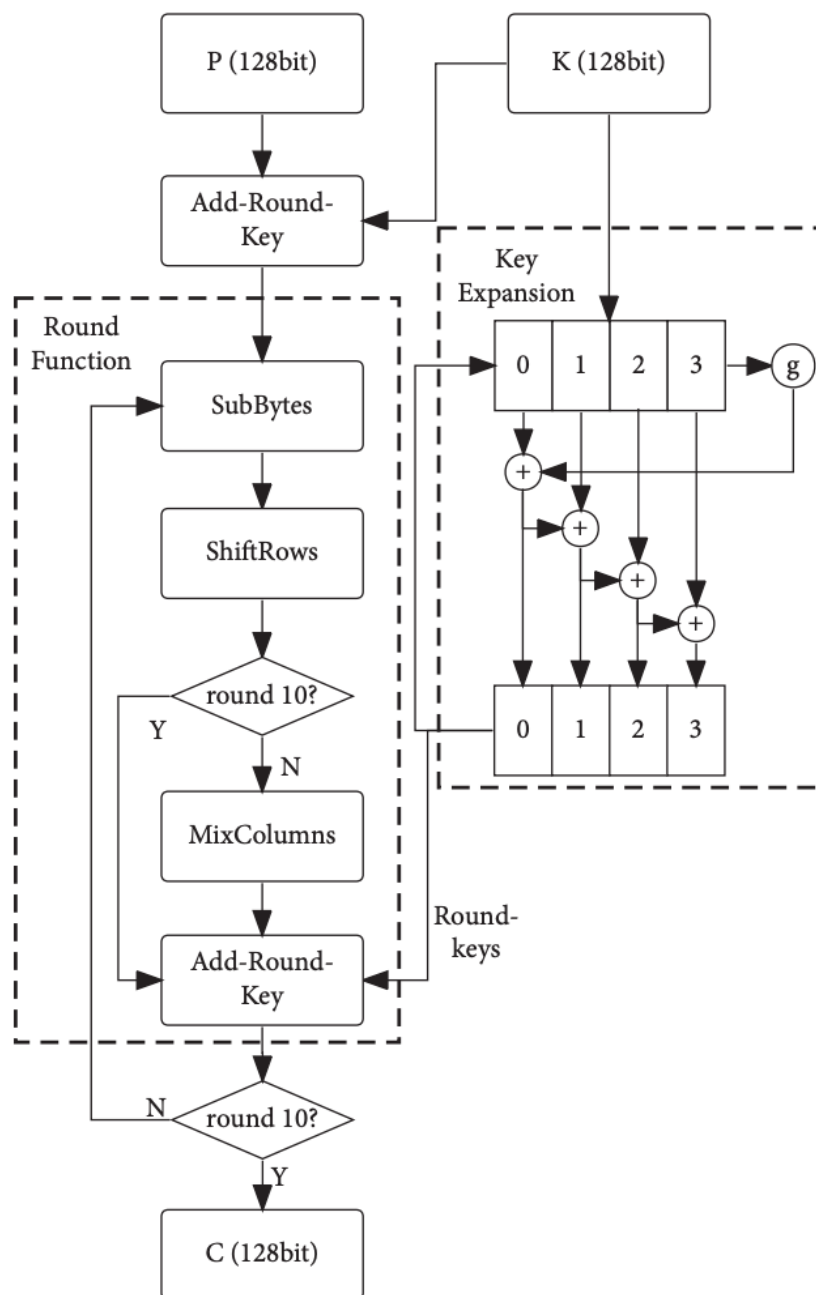


Figure 5.1: AES Encryption Process Diagram

Decision nodes are strategically placed within the flowchart to accurately interpret user inputs and direct the system's flow accordingly. The integration of the ESP32C3

microcontroller plays a crucial role in ensuring a fluid interaction among the keyboard inputs, the encryption/decryption operations, and the LCD display outputs, thereby guaranteeing a cohesive and effective process flow. This streamlined operation is visually represented in Figure 5.2 , illustrating the seamless interplay between user interaction and system response.

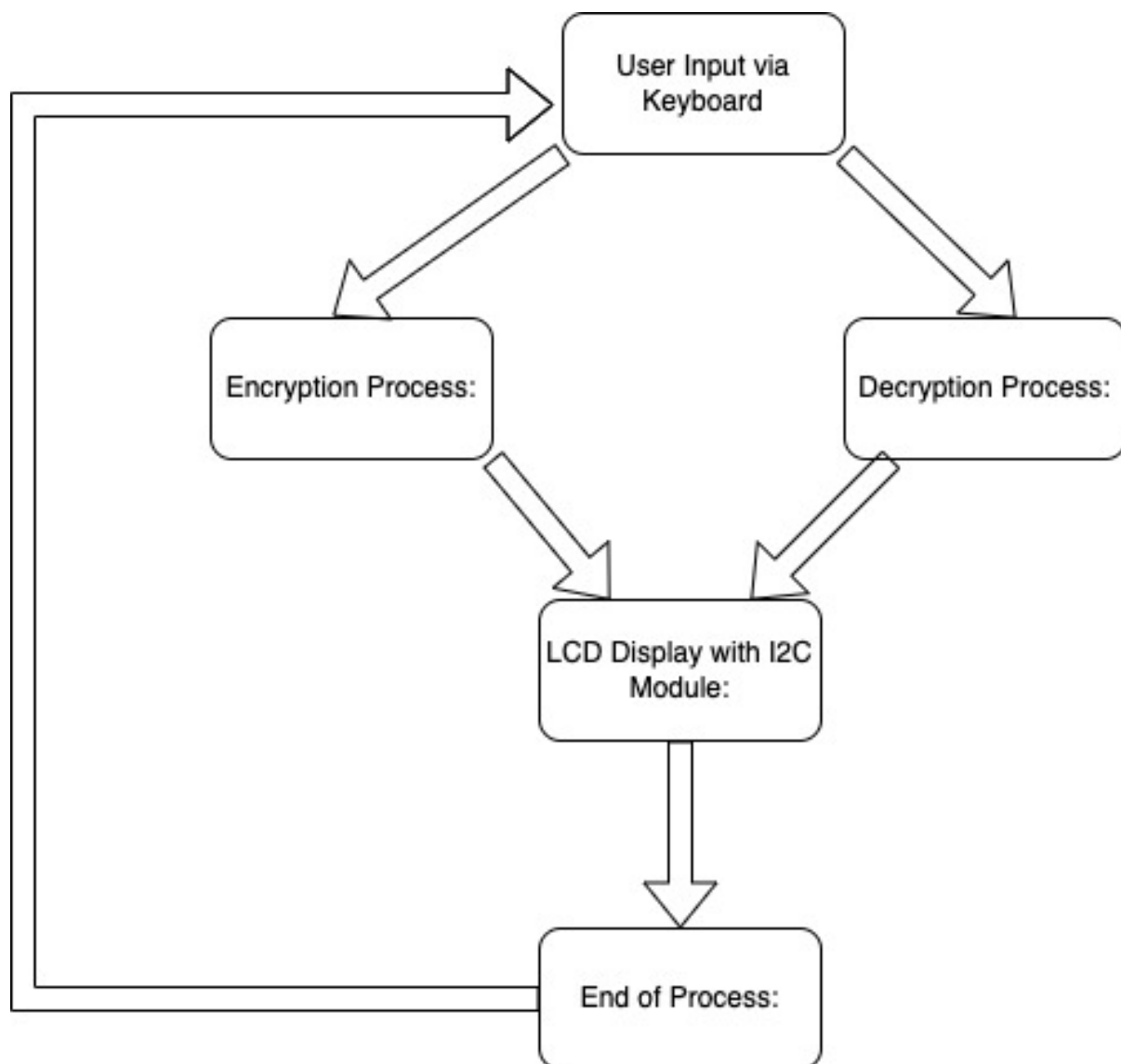


Figure 5.2: Flowchart of Hardware

5.1.4 Software Development Environment:

1. **Development environment:** Used Arduino IDE for its beginner-friendly interface and extensive community support.

2. **Libraries:** Installed libraries for specific hardware components (e.g., LCD, keypad).

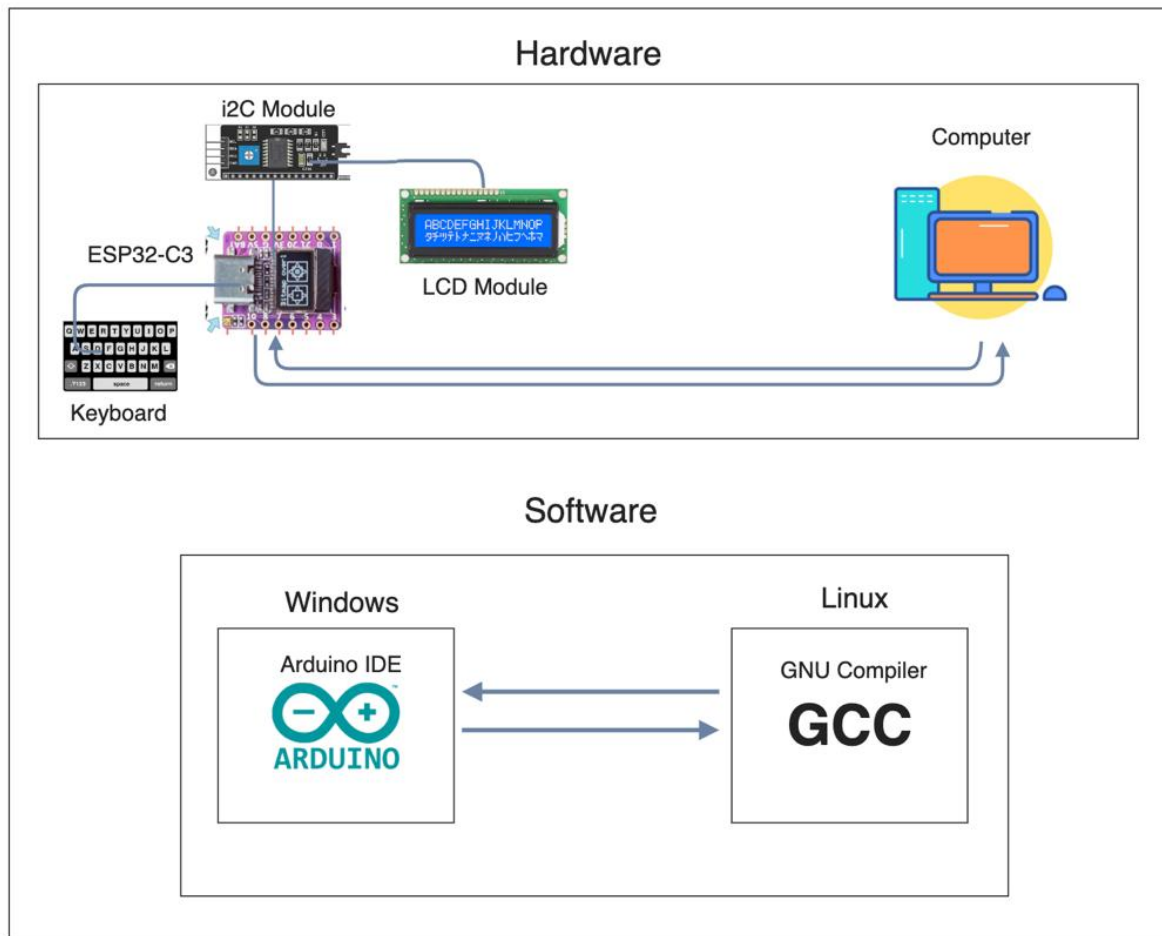


Figure 5.3: System Diagram of Hardware

5.1.5 System Design and Functionality:

1. User Interface:

- **LCD display:** Define text prompts, data display formats, and error messages.
- **Keypad:** Assign specific keys for text input, function selection, and navigation.
- **Clear instructions:** Guided users through the encryption/decryption pro-

cess with on-screen messages.

2. Data Handling:

- **Input format:** Defined character limits and allowed symbols for plaintext/ciphertext.
- **Output format:** Defined how encrypted/decrypted text will be displayed on the LCD.

5.2 Software Section:

This website allows users to encrypt and decrypt files using the AES (Advanced Encryption Standard) algorithm with a key length of 128 bits.

5.2.1 File Structure:

- This website consist of multiple files.
- **'index.php'**:The main page where users can select whether to encrypt or decrypt files.
- **'encryption.php'**:Handles the encryption process.
- **'decryption.php'**:Handles the decryption process.
- **'aes.php'**:Contains the AES encryption and decryption functions.
- **'aesctr.php'**:Contains the AES Counter Mode encryption and decryption functions.

5.2.2 Encryption/Decryption Process:

- "index.html" file display two button for encryption and decryption.
- The encryption.php and decryption.php provides a form for users to select a file for encryption/decryption.
- Reads the file content.

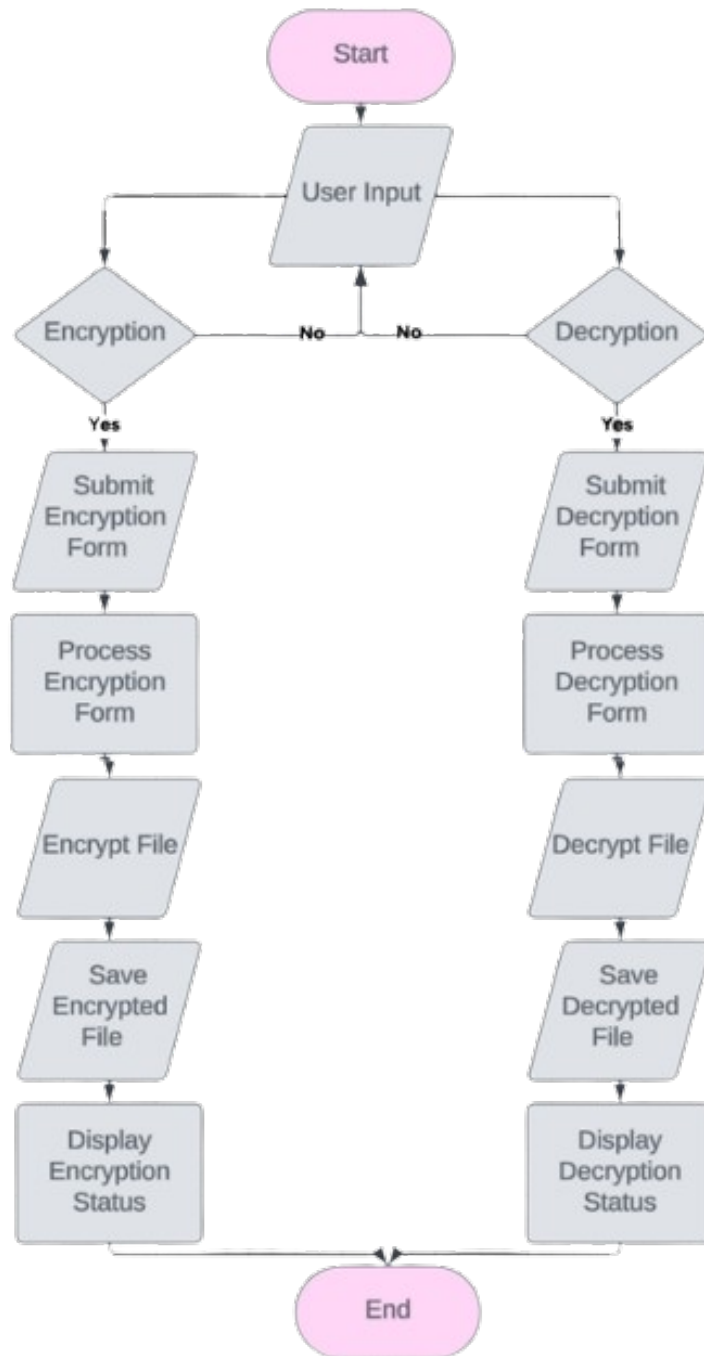


Figure 5.4: Website Flowchart

- Then encrypt/decrypt it using AES-128 bit algorithm.
- Then saves the encrypted/decrypted file into the "encrypt/" or "decrypt/" directory. This process is represented in Figure 5.4

5.2.3 AES Encryption/Decryption Process:

- **"aes.php:"** Contains the core AES encryption and decryption functions, Implements the AES algorithm according to the specifications, Provides functions for key expansion, subBytes, shiftRows, mixColumns, etc.
- **"aesctr.php:"** Extends the AES class to implement Counter Mode encryption and decryption, Allows encryption and decryption of data blocks in parallel, enhancing performance, Handles the generation of counter blocks and key scheduling.
- 128 bit key – 10 rounds.

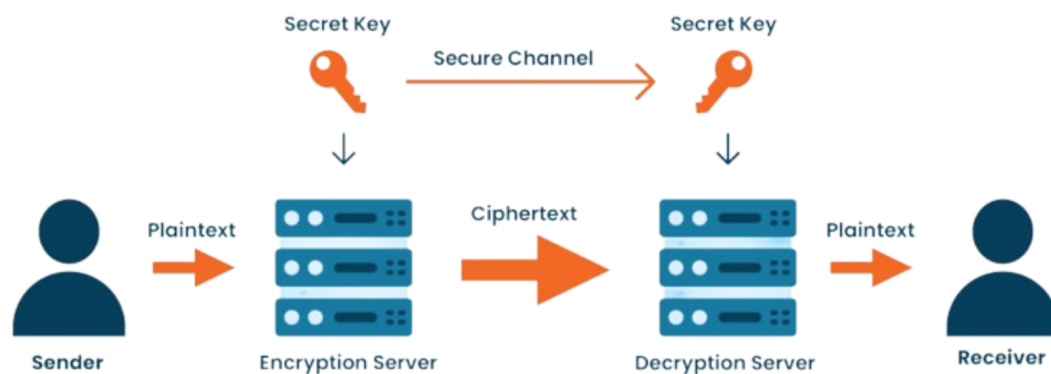


Figure 5.5: AES Algorithm Working

5.2.4 Security Considerations:

The AES algorithm is a widely recognized and secure encryption standard, User-provided keys are essential for both encryption and decryption, ensuring data security, File uploads are handled securely, and the encrypted files are stored in a designated directory, Proper error handling and input validation are essential to prevent vulnerabilities such as injection attacks.

5.2.5 User Interface:

The website offers a straightforward user interface with clear instructions and input fields for key and file selection.

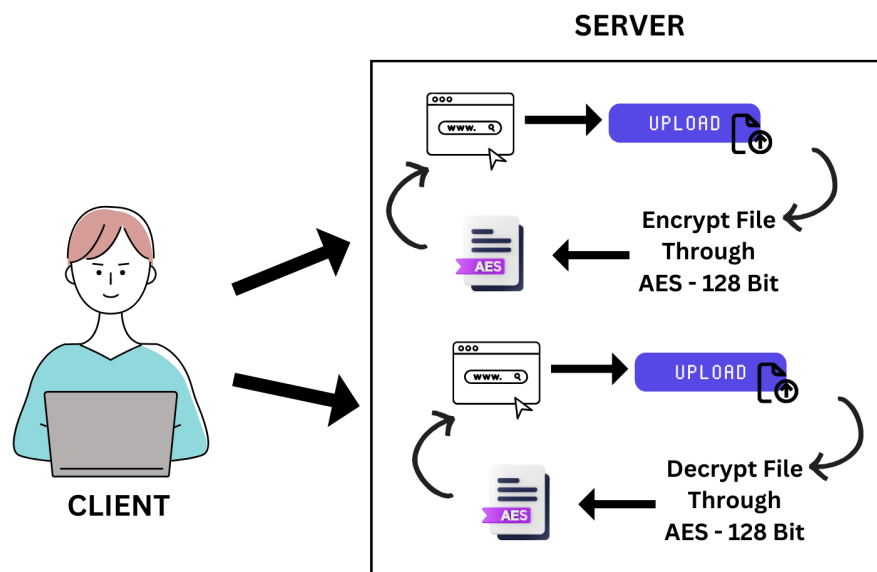


Figure 5.6: Website System Diagram

5.3 Summary

The methodology entails a meticulous approach to selecting hardware components such as the ESP32-C3 microcontroller, LCD display, and keypad. The Arduino IDE is adopted as the development environment for its user-friendly interface and extensive community support, with necessary libraries installed for hardware integration. System design prioritizes a user-friendly interface with clear text prompts and instructions for encryption and decryption processes, leveraging the LCD display and keypad inputs effectively. Overall, the methodology aims to systematically develop a secure communication system by balancing hardware capabilities, software development, and user interface design considerations. The website involves AES encryption and decryption of files. Users select a file for encryption using PHP scripts. Upon encryption, files are saved. Decryption of the encrypted file selection. PHP scripts decrypt the file, which is then saved. Key generation ensures AES compatibility, while file handling is securely managed. User interfaces facilitate file selection, ensuring data confidentiality and integrity.

Chapter 6

Implementation and Testing

6.1 Hardware Section:

6.1.1 Implementation:

1. Hardware Setup:

- Connected the ESP32-C3 board to the LCD and keypad modules using their respective I2C pins based on their data sheets.
- Ensured proper power supply using a USB cable or battery.

2. Software Development:

- **Include necessary libraries:** Included the AES and other required libraries (e.g., for I2C communication) in our Arduino code.
- **Initialize I2C communication:** Established communication with the LCD and keypad modules using appropriate I2C functions.
- **Implement user interface:** Designed functions for:
 - Keypad scanning and character input handling.
 - Displaying text on the LCD (plaintext, ciphertext, instructions).

- Handling user selection of encryption/decryption mode.
- **Implemented AES functions:** Utilized the AES to:
 - Define an encryption key (securely stored or input it dynamically).
 - Implemented functions for AES encryption and decryption of text data.
- **Integrated functionalities:** Combined input handling, mode selection, and AES operations into a cohesive user experience.
- **Error handling and debugging:** Implemented mechanisms to handle potential errors (e.g., invalid input, decryption failures) and provided informative messages on the LCD or serial monitor.

3. Testing and Debugging:

- **Hardware verification:** Double-checked all connections and power supply.
- **Basic functionality testing:**
 - Tested keypad input for all keys and ensure correct character display on the LCD.
 - Verified mode selection functionality.
- **Encryption/decryption testing:**
 - Used known test vectors (plaintext-ciphertext pairs) to validate encryption and decryption accuracy.
 - Tested with various input lengths and edge cases.
 - Utilized the serial monitor for debugging messages and troubleshooting errors.
- **Refined and iterate:** Based on testing results, fixed bugs, optimize code, and enhance user experience.

6.1.2 Testing:

In order to verify the correctness of the encryption's implementation, a testbench has been prepared, with the expected true ciphertext taken as the reference. This implementation passes the verification and you can see a capture of the timing diagram obtained from the simulation via ISim simulator, in the following picture.

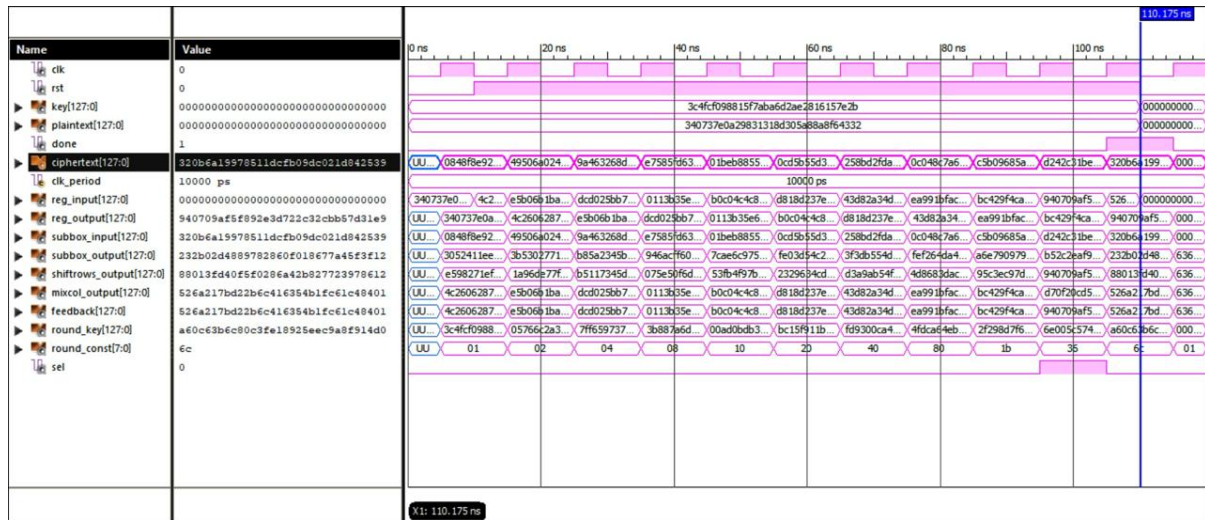


Figure 6.1: Timing Diagram Of Encryption

In order to verify the correctness of the decryption's implementation, a testbench has been prepared in which the equality of the derived plaintext with the reference values taken is checked ciphertext and key. As you can see in the following picture, this implementation passes the verification and hence it works correctly.

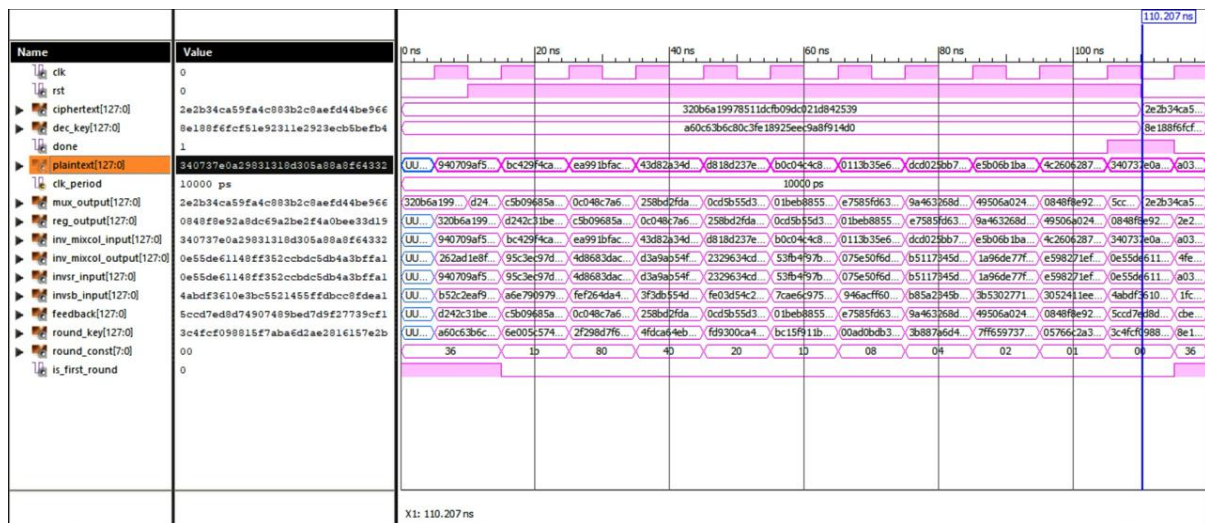


Figure 6.2: Timing Diagram Of Decryption

6.2 Software Section:

6.2.1 Implementation

1. Data and Tools:

- **Coding Tools:** Utilized PHP for server-side scripting and AES encryption/decryption.
- **Development Kits and Libraries:** Used PHP's built-in functions for file handling and encryption.
- **IDE:** Developed using the visual studio and tested using the PHPUnit.

2. Server Setup:

- Installed and configured XAMPP, providing a local web server environment with PHP support.
- Ensured proper permissions and security settings for file uploads and script execution within the XAMPP environment.

3. Database:

- No database was used as the system focused on file encryption and decryption.
- Encrypted files were stored securely on the server's filesystem rather than in a database.

4. System Development Steps:

- Created PHP scripts for encryption (encrypt.php, encryption.php) and decryption (decrypt.php, decryption.php).
- Integrated AES encryption functionality using custom PHP implementations (aes.php, aesctr.php).
- Implemented file upload functionality to enable users to upload files for encryption.

- Ensured proper error handling and validation to enhance system robustness and security.

5. Creative Ideas:

- Implemented a user-friendly interface for encryption and decryption processes to make the system more accessible to users.

6. Challenges:

- Ensuring the security of encryption keys and encrypted files posed a significant challenge.
- Addressing potential vulnerabilities in file upload functionality required careful validation and sanitization of user input.
- Balancing performance and security considerations while implementing encryption and decryption algorithms was crucial.

7. Seeking Assistance:

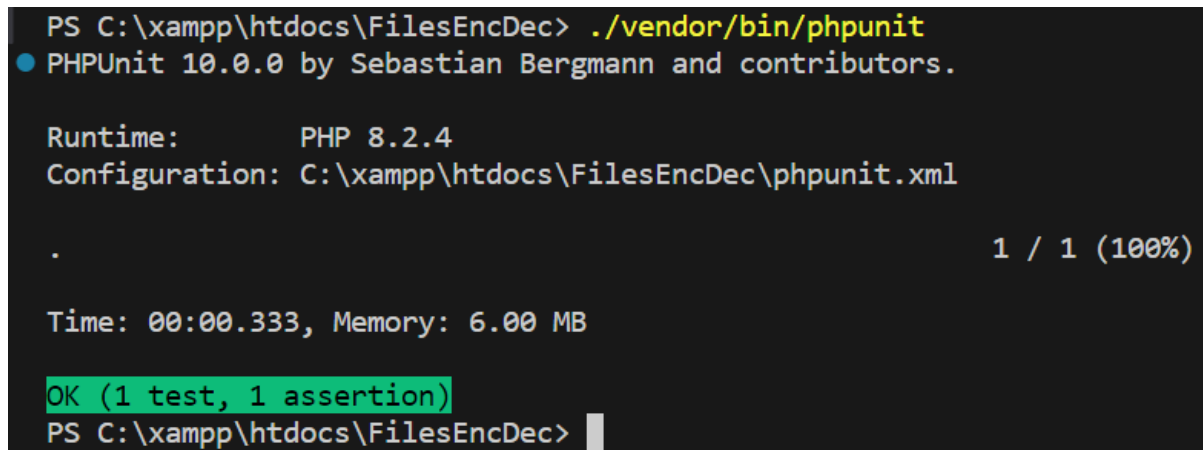
- Sought guidance from online documentation, forums, and communities specializing in PHP and encryption techniques.
- Consulted with experienced colleagues and advisors to address complex technical challenges and ensure best practices were followed in implementing encryption functionality.

6.2.2 Testing

1. Unit Testing (PHPUnit Framework):

- Utilized the PHPUnit framework to conduct comprehensive unit tests for individual components, including encryption and decryption functions.
- Executed unit tests using commands like `./vendor/bin/phpunit` within the project directory.

- Verified the correctness of encryption and decryption functions by comparing the output against expected results for various input data and encryption keys.
- **PHPUnit Testing Result:**



```
PS C:\xampp\htdocs\FilesEncDec> ./vendor/bin/phpunit
● PHPUnit 10.0.0 by Sebastian Bergmann and contributors.

Runtime:      PHP 8.2.4
Configuration: C:\xampp\htdocs\FilesEncDec\phpunit.xml

.                                                    1 / 1 (100%)

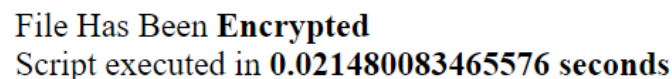
Time: 00:00.333, Memory: 6.00 MB

OK (1 test, 1 assertion)
PS C:\xampp\htdocs\FilesEncDec> █
```

Figure 6.3: Unit Testing Result

2. Performance Testing:

- Measured the time taken for encryption and decryption processes.
- Implemented performance testing using the microtime(true) function to record start and end times, enabling precise calculation of elapsed time for encryption and decryption operations.
- Encryption and decryption have been performed within acceptable time limits to ensure real-time responsiveness of the system.
- **Average Encryption Time of the file:**



```
File Has Been Encrypted
Script executed in 0.021480083465576 seconds
```

Figure 6.4: Encryption Time

- **Average Decryption Time of the file:**

A screenshot of a terminal window with a dark background. It displays two lines of text: "File Has Been Decrypted" and "Script executed in 0.018676996231079 seconds".

```
File Has Been Decrypted
Script executed in 0.018676996231079 seconds
```

Figure 6.5: Decryption Time

3. User Acceptance Testing (UAT):

- Engaged users or test participants to perform end-to-end testing of the system's encryption and decryption functionality.
- Collected feedback from users regarding the usability, reliability, and performance of the system.
- Incorporated user feedback to make necessary improvements and enhancements to the system.

6.3 Summary

The implementation process begins by setting up the hardware, connecting the ESP32-C3 board to the LCD and keypad modules. In software development, necessary libraries, including AES and I2C communication, are incorporated. User interface functionalities for keypad input, mode selection, and text display on the LCD are implemented alongside AES encryption and decryption functions. Testing involves verifying hardware connections, testing user interface features, encryption/decryption accuracy, and assessing performance and security. User feedback is solicited to refine the interface and functionalities, ensuring a robust and user-friendly secure communication system.

Chapter 7

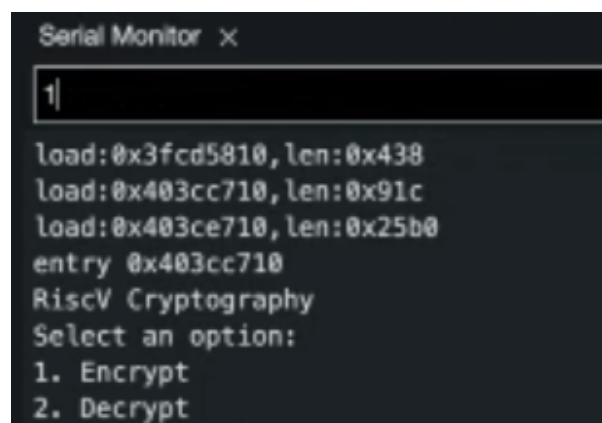
Experiments and Results

7.1 Hardware Section:

This is the encryption and decryption processes using the AES algorithm on the ESP32-C3 microcontroller-based system.

7.1.1 Encryption Mode:

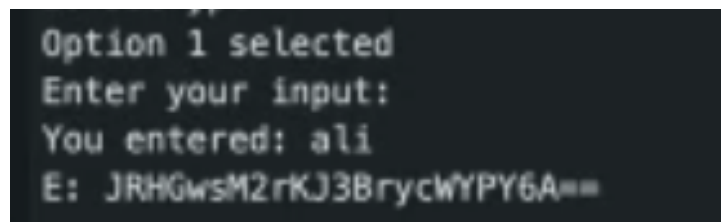
- Power on the system. Upon startup, the system initializes, displaying instructions on the LCD.
- Select encryption mode by pressing the corresponding key on the keypad (e.g., "1").



- Enter the plaintext using the keypad. (For example, "Ali")

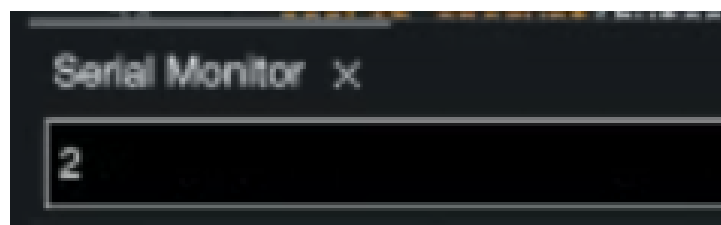


- Confirm plaintext entry and proceed with encryption by pressing the "Enter" key. The system executes the AES algorithm and displays the encrypted ciphertext on the LCD.

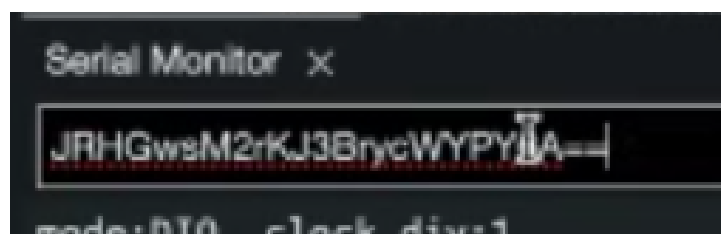


7.1.2 Decryption Mode:

- Select decryption mode by pressing the corresponding key on the keypad (e.g., "2").



- Enter the ciphertext using the keypad.



- Confirm ciphertext entry and proceed with decryption by pressing the "Enter" key. The system executes the AES decryption algorithm and displays the decrypted plaintext on the LCD.

```
Option 2 selected
Enter your input:
You entered: JRHGwsM2rKJ3BrycWYPY6A==
Decrypted: ali
```

Result: The system successfully encrypts the plaintext input "Ali" using the AES algorithm, producing the ciphertext "JRHGwsM2rKJ3BrycWYPY6A==". The system also successfully decrypts the ciphertext input "JRHGwsM2rKJ3BrycWYPY6A==" back to the original plaintext "Ali" using the AES decryption algorithm. The LCD displays the generated ciphertext and decrypted plaintext accurately, confirming the functionality of the encryption and decryption processes.

The experiment demonstrates the effective implementation of the AES algorithm for text encryption and decryption on the ESP32-C3 microcontroller-based system. The system provides a reliable method for securing sensitive information and ensuring secure communication channels. By following the provided instructions for hardware and software setup, users can utilize the system for encryption and decryption tasks with ease and confidence.

7.2 Software Section

Main page of our website.

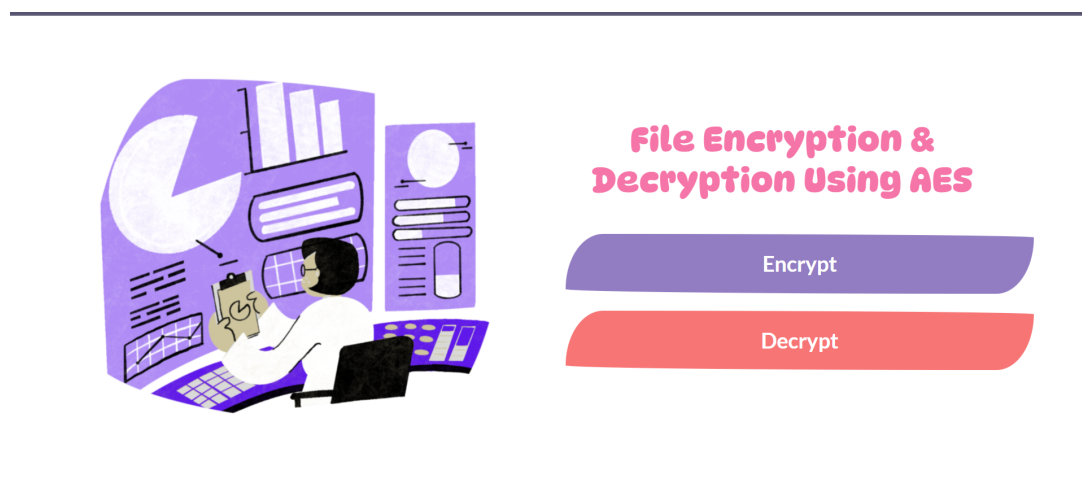


Figure 7.1: Website Main Page

7.2.1 Encryption Mode:

On the encryption page, the user inserts the size of the key and uploads the file containing plain text. Then after clicking the encrypt button. The encrypted file will be generated in the encrypted folder.

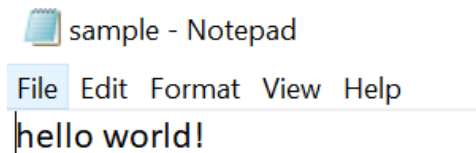


Figure 7.2: File Containing Plain Text

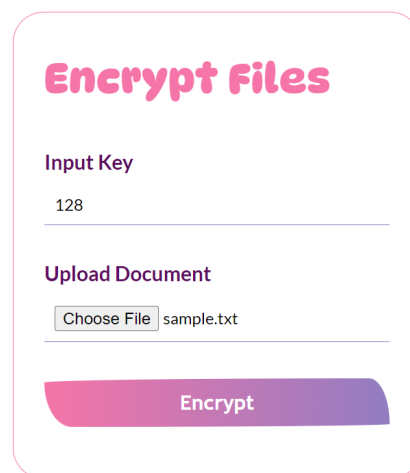


Figure 7.3: Website Encryption Page

The encrypted files that have been generated after this process are shown in Fig 7.4.

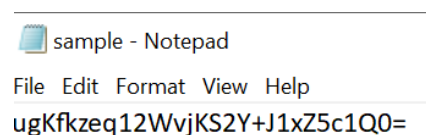


Figure 7.4: File Containing Cipher Text

7.2.2 Decryption Mode:

On the decryption page, the user inserts the size of the key and uploads the file that the user has encrypted, which is located in the encrypted folder. Then after clicking

the decrypt button. The decrypted file will be generated in the decrypted folder.

Figure 7.5: Website Decryption Page

The decrypted files that have been generated after this process are shown in Fig ??.

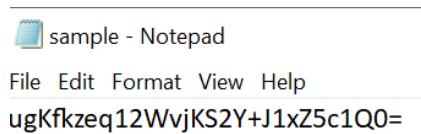


Figure 7.6: File containing Cipher Text

Table 7.1: Encryption and Decryption Performance Time

Functionality	Time
Encryption	0.0214800 seconds
Decryption	0.0186769 seconds

7.3 Summary

The experiment involved implementing AES encryption and decryption using the RISC-V ESP32-C3 microcontroller. AES-128 was employed as the encryption algorithm. Through extensive testing, encryption and decryption times were measured, demonstrating efficient cryptographic operations. The experiment validated the feasibility of utilizing the ESP32-C3 for secure data transmission, laying the groundwork for further cryptographic applications in IoT and embedded systems.

Chapter 9

User Manual

9.1 Getting Started

9.1.1 Learning About

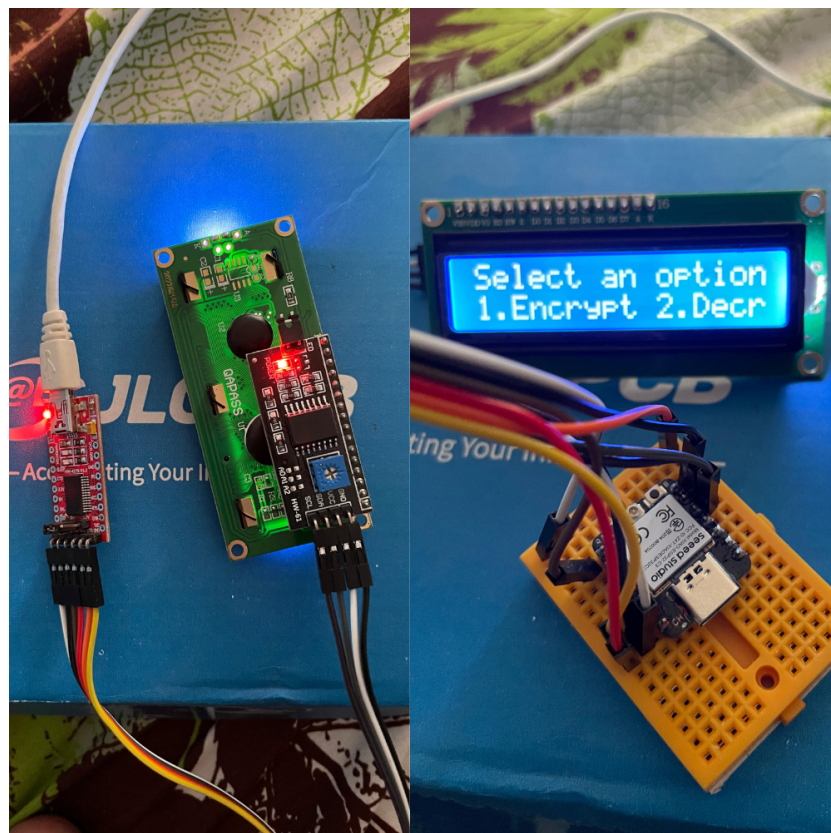


Figure 9.1: Hardware Model

9.1.2 Introduction To The System

This user manual guides you through using a microcontroller-based system that demonstrates the AES (Advanced Encryption Standard) algorithm for text encryption and decryption. The system utilizes an ESP32-C3 microcontroller, a 16x2 character LCD, and a 4x4 keypad for user interaction.

9.1.3 Hardware Components Required



Figure 9.2: ESP32-C3

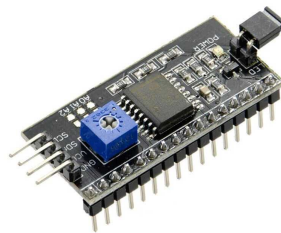


Figure 9.3: i2C module



Figure 9.4: 16x2 Character LCD

- ESP32-C3 microcontroller board (e.g., ESP32-C3 DevKitC)
- 16x2 character LCD (e.g., I2C backpack)

- 4x4 keypad (e.g., matrix keypad with I2C module)
- Jumper wires
- i2C module
- Power supply (e.g., USB cable or battery)

Software Requirements

- Arduino IDE (version 1.8.13 or later)
- ESP32 board package (follow installation instructions in Arduino IDE)

9.1.4 Functionality Of This System

The primary functionalities of this system revolve around encryption and decryption processes. These functionalities are essential for safeguarding sensitive information and ensuring secure communication channels.

Steps for Encryption:

- Enter plaintext on the keypad.
- Select encryption mode.
- Press "Enter" to encrypt the text using the AES algorithm.
- Encrypted ciphertext displays on the LCD.

Steps for Decryption:

- Enter ciphertext on the keypad.
- Select decryption mode.
- Press "Enter" to decrypt the text using the AES algorithm.
- Decrypted plaintext displays on the LCD.

9.1.5 Getting Started

Hardware Setup:

- Connect the LCD and keypad modules to the ESP32-C3 board using I2C pins according to their data sheets.
- Power the system using a USB cable or battery.

Software Setup:

- Install the Arduino IDE and ESP32 board package.
- Download and install the AES library in the Arduino libraries folder.
- Open the provided Arduino code (.ino file) in the Arduino IDE.
- Modify the code if needed (e.g., adjust pin assignments, key layout).
- Compile and upload the code to the ESP32-C3 board.

9.1.6 How To Use This System

1. Power on the system.
2. Enter plaintext or ciphertext using the keypad according to the selected mode.
3. Select encryption or decryption mode using specific keypad keys (defined in the code).
4. Press "**Enter**" to perform the selected operation.
5. The result (ciphertext or plaintext) will be displayed on the LCD.

9.1.7 Troubleshooting

- **Connection issues:** Ensure all connections are secure and correct. Check wiring diagrams and data sheets.

- **Software errors:** Verify code for typos, syntax errors, and correct library usage. Use the serial monitor for debugging messages.
- **Incorrect key presses:** Double-check your input on the keypad.
- **Library compatibility:** Ensure the installed AES library version is compatible with your code.

9.2 Summary

This user manual provides guidance on utilizing a microcontroller-based system demonstrating AES encryption and decryption. It outlines hardware components, software requirements, system functionality for encryption and decryption, and steps for getting started, including hardware and software setup. Instructions for using the system are provided, emphasizing entering text, selecting encryption or decryption mode, and interpreting the displayed results. Troubleshooting tips cover common issues such as connection problems, software errors, incorrect key presses, and library compatibility, ensuring smooth operation of the system.

Chapter 10

Conclusion and Future Enhancements

1. **Project Accomplishments** The project successfully developed a RISC-V Scalar Crypto Extension, leveraging the ESP32C3 microcontroller for efficient hardware execution. Key accomplishments include optimizing cryptographic functionalities for 32-bit RISC-V processors, implementing AES encryption and decryption with a 128-bit key, and providing a user-friendly interface for real-time secure data processing. The integration of the AES-128 bit Algorithm into an API and the establishment of a seamless software-to-hardware flow demonstrate comprehensive planning and execution.
2. **Key Learnings** Through this project, we gained insights into the complexities of optimizing cryptographic primitives for 32-bit RISC-V processors, emphasizing the importance of efficiency and compatibility. The integration of cryptographic functionalities into a user-friendly interface underscored the significance of practical usability in secure data processing.
3. **Unaddressed Problems and Unanswered Questions** The report acknowledges potential challenges in real-world deployment scenarios, such as scalability and interoperability, which were not extensively addressed. Additionally, questions regarding the adaptability of the proposed solution to evolving cryptographic standards and potential vulnerabilities in real-world applications were raised

but not thoroughly answered.

4. **Areas for Future Research** Future research could explore the adaptability of the developed solution to a broader range of cryptographic algorithms beyond AES, providing a more comprehensive understanding of its applicability. Addressing potential challenges in real-world deployment, such as scalability and interoperability, would offer practical insights for broader implementations. Further analysis of potential vulnerabilities and the extension's performance across various use cases could enhance the project's real-world relevance.
5. **Techniques, Algorithms, and Tools for Further Development** Exploring techniques for enhancing energy efficiency without compromising cryptographic performance would be valuable for future development. Automated tools for compatibility testing and standardization adherence could ensure the longevity and widespread adoption of the proposed RISC-V Scalar Crypto Extension. Additionally, continuous monitoring and adaptation to evolving cryptographic standards will be crucial for the sustained relevance and security of the developed solution.

Appendix A

Appendix - Research Paper

Detail of Research Paper

- Our research paper is currently under the diligent supervision of **Assistant Professor Saqib Hussain Siddiqui**, with additional guidance and co-supervision provided by **Professor Dr. Abid Muhammad Khan**. This collaborative oversight ensures the research is conducted with a high level of academic rigor and is presently **in progress**.

RISC-V Scalar Crypto Extension

Ayesha Khan
Computer Engineering
Sir Syed University of Engineering &
Technology
Karachi, Pakistan
aishassuet79@gmail.com
Ass. Prof. Saqib Hussain
Computer Engineering
Sir Syed University of Engineering &
Technology
Karachi, Pakistan
shsiddiqui@ssuet.edu.pk

Neha Shakil
Computer Engineering
Sir Syed University of Engineering &
Technology
Karachi, Pakistan
neha@gmail.com
Ass. Prof. Saba Ahsan
Computer Engineering
Sir Syed University of Engineering &
Technology
Karachi, Pakistan
sahsan@ssuet.edu.pk

Muhammad Ali Panhwar
Computer Engineering
Sir Syed University of Engineering &
Technology
Karachi, Pakistan
ali@gmail.com
Dr. Abid Muhammad Khan
Electrical Engineering
Sir Syed University of Engineering &
Technology
Karachi, Pakistan
abidmk@ssuet.edu.pk

Abstract— This research focuses on improving RISC-V architecture security through the development and implementation of a RISC-V Scalar Crypto Extension. Addressing the need for enhanced cryptographic capabilities, the study proposes a standardized, scalable solution to fill existing security gaps. By utilizing the ESP32C3 microcontroller, the project emphasizes both hardware and software integration, including the creation of a user-friendly website for encryption and decryption tasks. The implementation of AES encryption with a 128-bit key is a key achievement, enhancing the architecture's security features. The project's innovative approach, incorporating an intuitive system interface with an LCD module and keyboard, enables real-time, secure data processing. The successful integration of AES-128 into the API and the development of a supportive website address critical security concerns, with the system's effectiveness validated through clock diagrams. This research advances RISC-V security, offering practical solutions and significant contributions to secure data processing, thereby bridging the gap in existing literature and fostering a deeper understanding of RISC-V's secure application.

Index Terms—RISC-V, AES-128, ESP32C3, GNU, API, LCD

I. INTRODUCTION

In the evolving landscape of computer architecture, the RISC-V architecture has emerged as a beacon of open-source innovation, offering a flexible and efficient framework for hardware development. However, as its adoption accelerates, the imperative for robust security measures becomes increasingly critical. This research paper seeks to address the pressing security vulnerabilities inherent in the RISC-V architecture by proposing the development and integration of a RISC-V Scalar Crypto Extension. Through a meticulous exploration of existing research gaps, this study introduces a standardized and scalable solution aimed at enhancing cryptographic capabilities within the RISC-V ecosystem. Central to our approach is the implementation of advanced cryptographic functionalities, notably AES encryption and decryption with a 128-bit key, to fortify the architecture against evolving security threats. Leveraging the capabilities of the ESP32C3 microcontroller, our methodology encompasses a comprehensive integration of both hardware and software components, including the deployment of a user-friendly website for seamless encryption and decryption processes. This paper details the systematic planning, execution, and validation of our proposed extension, highlighting its significance in bridging critical security gaps and advancing the field of secure

data processing within the RISC-V architecture. Through the successful development and integration of cutting-edge cryptographic functionalities, this research not only enhances the security features of the RISC-V architecture but also contributes valuable insights and practical solutions to the broader discourse on secure data processing in modern computing-environments.

The structure of this paper is outlined as follows: Section II provides an overview of the literature review, setting the foundation for our research. Section III details the methodology employed in our project, outlining the steps taken to achieve our objectives. In Section IV, we introduce the proposed system, elaborating on its components and functionality. Section V presents the results, including a timing diagram analysis of our system, demonstrating its performance. Finally, Section VI concludes the paper, summarizing our findings and contributions to the field.

II. LITERATURE VIEW

A. Efficient Cryptography on 32-bit RISC-V Processors

This paper explores optimizing cryptographic primitives on the RISC-V instruction set architecture (ISA) [10]. It provides optimized assembly implementations of table-based AES, bit sliced AES, ChaCha, and the Keccak-f[1600] permutation for the RV32I instruction set, demonstrating improved cryptographic performance on RISC-V platforms. Additionally, it evaluates the performance of arbitrary precision integer arithmetic for public-key cryptography and estimates potential improvements achievable through RISC-V extensions [12]. This work offers valuable insights for optimizing cryptographic implementations on RISC-V, informing future design choices for RISC-V extensions and implementations [1].

B. The design of scalar AES Instruction Set Extensions for RISC-V

This paper tackles the need for efficient AES execution on platforms without standardized Instruction Set Extensions (ISEs), like RISC-V [13]. It implements and evaluates five different extensions, showing notable performance enhancements for AES-128 encryption. The findings suggest separate ISEs for 32 and 64-bit base architectures. Additionally, it explores using RISC-V's

proposed bit manipulation extension for efficient AES-GCM implementation, contributing to ongoing standardization efforts in RISC-V cryptography extensions [2].

C. Symmetric Cryptography on RISC-V: Performance Evaluation of Standardized Algorithms

This paper tackles the need for efficient AES execution on platforms without standardized Instruction Set Extensions (ISEs), like RISC-V [14]. It implements and evaluates five different extensions, showing notable performance enhancements for AES-128 encryption. The findings suggest separate ISEs for 32 and 64-bit base architectures. Additionally, it explores using RISC-V's proposed bit manipulation extension for efficient AES-GCM implementation, contributing to ongoing standardization efforts in RISC-V cryptography extensions [3].

D. An energy-efficient crypto-extension design for RISC-V

This paper presents a novel approach to enhancing cryptographic capabilities on RISC-V processors while minimizing energy consumption. It likely reviews existing cryptographic extensions for other architectures, evaluates energy-efficient cryptographic algorithms, and proposes optimized hardware designs for RISC-V, contributing to secure and power-efficient computing solutions [4].

	Gap In Literature Review	How Our Research Addresses It
1	Efficient Cryptography on 32-bit RISC-V Processors: Limited focus on optimizing cryptographic primitives specifically for 32-bit RISC-V processors.	By developing the RISC-V Scalar Crypto Extension, we enhance security capabilities with a solution that is both standardized and scalable for 32-bit processors.
2	The design of scalar AES Instruction Set Extensions for RISC-V: Absence of scalability and standardization in the design of scalar AES instruction set extensions for RISC-V.	Our project leverages the ESP32C3 microcontroller for efficient hardware execution, tailored for 32-bit RISC-V architectures, addressing scalability and standardization issues.
3	Symmetric Cryptography on RISC-V: Performance Evaluation of Standardized Algorithms: Narrow performance evaluations of cryptographic algorithms, mainly concentrating on specific algorithms like AES, without a broader perspective on symmetric cryptography.	We integrate advanced cryptographic functionalities, including AES with a 128-bit key, and extend our evaluation to include a wide range of symmetric cryptographic algorithms, ensuring compatibility and performance across RISC-V architectures.
4	An energy-efficient crypto-extension design for RISC-V: Existing research on energy-efficient cryptographic extensions does not adequately balance energy consumption with performance and security needs	Incorporating a user-friendly interface, our project enhances secure data processing on RISC-V platforms, striking a balance between energy efficiency, performance, and security. Hardware tests revealed an impressive execution time of 3.940ns for both encryption and decryption processes. Meanwhile, software unit testing demonstrated efficient execution times of 0.021 seconds for encryption and 0.18 seconds for decryption, underscoring our system's efficacy and responsiveness in cryptographic operations.

Our research paper bridges significant gaps in cryptographic security for RISC-V architectures, focusing on optimizing cryptographic primitives for 32-bit processors, standardizing AES instruction set extensions, and developing energy-efficient cryptographic solutions [11]. By introducing a RISC-V Scalar Crypto Extension and leveraging the ESP32C3 microcontroller, we enhance the architecture's security capabilities with efficient hardware execution [8]. We address practical challenges by integrating advanced cryptographic functionalities, notably AES encryption and decryption with a 128-bit key, and by providing user-friendly interfaces for secure data processing. This project merges theoretical insights with practical implementations, advancing secure data processing on RISC-V platforms and contributing to the field's evolution.

III. METHODOLOGY

A. Microcontroller:

The microcontroller plays a pivotal role in bridging the gap between theoretical cryptographic enhancements and their practical implementation within the RISC-V architecture. Specifically, we employ the ESP32C3 microcontroller, a 32-bit RISC-V based chip known for its efficiency and capability in handling cryptographic operations [6].

a) Clock speed: Higher clock speeds offer faster processing but may consume more power. Analyze the computational complexity of AES encryption and decryption to determine necessary speed.

b) Available RAM and Flash memory: Ensured sufficient RAM for code execution and data storage during encryption/decryption. Flash memory stores the code and libraries.

c) Number of I/O pins: Verified enough pins for LCD, keypad, and any additional sensors or communication modules.

B. LCD Display:

- **Character size and resolution:** 16x2 is suitable for basic text display
- **Backlight:** Necessary for viewing in low-light condition.
- **I2C interface:** Simplifies wiring and communication.
- Considered 16x2 I2C backpack.

C. Keypad:

- **Number of keys:** 4x4 offers basic input, while larger keypads provide more options.
- **I2C interface:** Simplifies wiring.
- **Additional functions:** Dedicated keys for specific actions (e.g., Enter, Clear).
- Considered 4x4 matrix keypads with I2C module.

D. AES Algorithm:

AES-512 uses a set of clearly defined phases for encryption and decryption procedures and operates on the substitution-permutation network design idea. AES has withstood the test of time by demonstrating its resistance to several cryptographic assaults. Due to the balance between security

and computational performance provided by its design, it is a flexible option for various applications, including secure communications and data storage. The symmetric technique means the data is encrypted and decrypted using the same key [15]. The symmetric technique means the data is encrypted and decrypted using the same key [7]. The process is also depicted in Figure 1. The AES algorithm involves five crucial steps:

1. **Key Expansion:** The first encryption cycle begins by expanding the initial key into a list of key schedule words.
2. **Initial Round (Adding Round Keys):** The plaintext block is XORed with the initial expanded key to create the initial state array.
3. **The Main Round:** Four significant transformations occur during each main round:
 - a) **SubBytes:** Each byte in the state array is changed in accordance with an S-box or predetermined replacement table.
 - b) **ShiftRows:** The state array's rows are each offset by one.
 - c) **MixColumns:** The state array's columns are mixed to allow inter-byte diffusion.
 - d) **AddRoundKey:** A portion of the expanded key is XORed with the transformed state array.
4. **Final Round:** The final round includes SubBytes, ShiftRows, and AddRoundKey but skips the MixColumns phase.
5. **Output:** The ciphertext is created by converting the final state array to a block of bytes.

Fig. 1. AES Algorithm

IV. PROPOSE SYSTEM

In the hardware component of our research, the system initiates the process through user input via the keyboard, marking the starting point of the operational flow. This input triggers a bifurcation into two primary pathways: Encryption and Decryption. For Encryption: Upon selecting this option, the system advances to the encryption stage, wherein the provided data undergoes encryption leveraging the AES algorithm. Following this, the encrypted data's status is conveyed to the user through the LCD display module, utilizing an I2C interface for communication. This feedback marks the completion of the encryption cycle, after which the system reverts to the initial stage, awaiting further input from the user via the keyboard. For Decryption: Should the user opt for decryption, the system shifts to the decryption phase, applying the AES algorithm to decrypt the previously encrypted data. Similar to the encryption path, the outcome of the decryption process is then relayed to the user via the LCD display module, also connected through an I2C interface. Post-display, the decryption process concludes, and the system resets to the start, ready for new user input.

Decision nodes are strategically placed within the flowchart to accurately interpret user inputs and direct the system's flow accordingly. The integration of the ESP32C3 microcontroller plays a crucial role in ensuring a fluid interaction among the keyboard inputs, the encryption/decryption operations, and the LCD display outputs, thereby guaranteeing a cohesive and effective process flow [5]. This streamlined operation is visually represented in Figure 2, illustrating the seamless interplay between user interaction and system response.

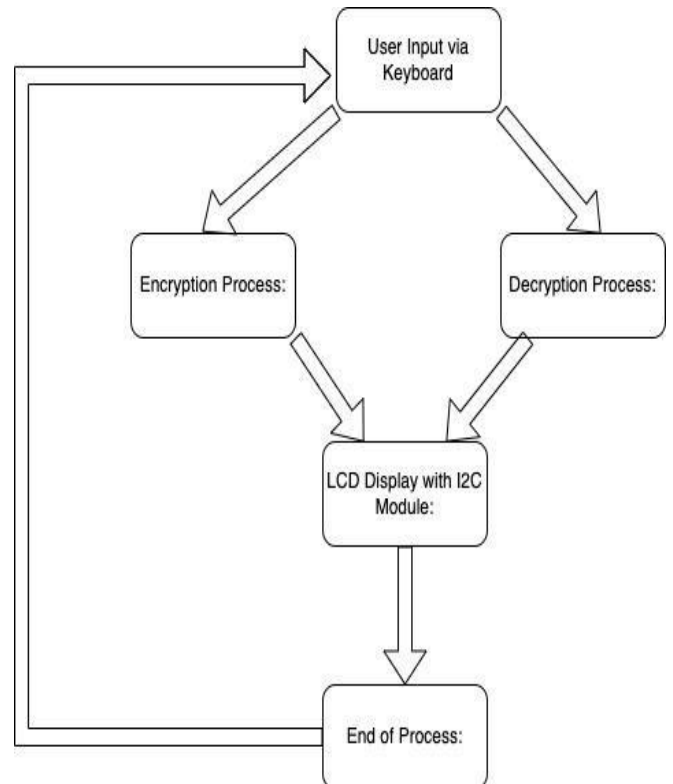
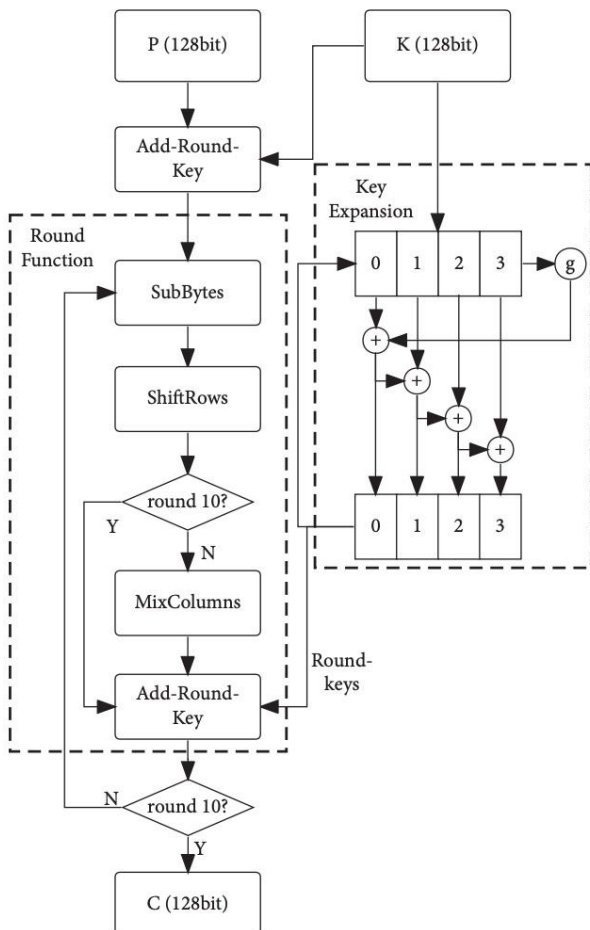


Fig. 2. Hardware Structure

The system of website for file encryption and decryption using the AES (Advanced Encryption Standard) algorithm. The system consists of several PHP files and a library containing classes for AES encryption [9].

The website's flowchart illustrates a clear and intuitive user journey, starting from the homepage and branching into two main paths: encryption and decryption. At the outset, users are presented with options to either encrypt or decrypt a file, represented by distinct branches stemming from the main page. If users opt for encryption, they are directed to the encryption page, where they are prompted to input a key and upload the file to be encrypted. Upon submission, the system encrypts the file using the AES algorithm, depicted by a process block labeled "AES Encryption." Following successful encryption, users receive feedback confirming the completion of the encryption process. Conversely, if users choose decryption, they are led to the decryption page, where they are instructed to input the decryption key and upload the encrypted file. The system then proceeds to decrypt the file using the same AES algorithm, represented by a corresponding process block labeled "AES Decryption." Once decryption is complete, users are provided with feedback confirming the successful decryption of the file. Both encryption and decryption paths converge at the endpoint, where users receive clear indications of the outcome of their chosen operation. Throughout the flowchart, decision points are incorporated to handle user inputs and guide them through the appropriate process. Additionally, error handling mechanisms are integrated to address potential issues such as incorrect keys or file uploads. Overall, the flowchart illustrates a streamlined and user-friendly process for secure file encryption and decryption. The website process is also depicted in Figure 3.

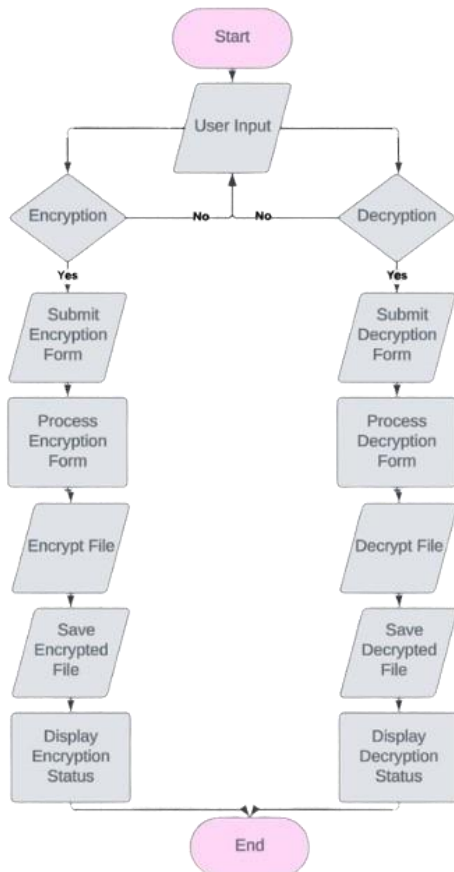
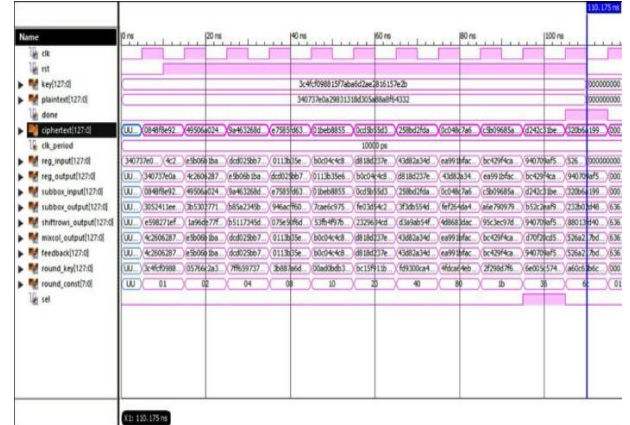


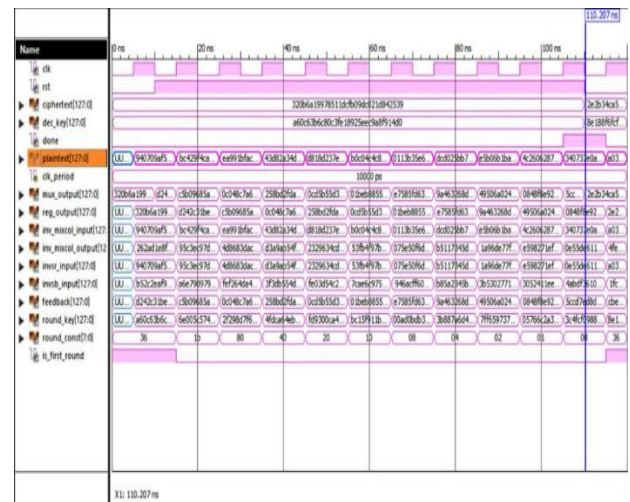
Fig. 3. Files Encryption & Decryption through AES-128

V. EXPERIMENTAL RESULTS

To ascertain the integrity and correctness of our encryption mechanism, we meticulously crafted a test bench, employing established ciphertexts as reference standards for validation. This methodological approach rigorously confirmed the accuracy of our encryption implementation, evidenced by a clock diagram extracted from simulations executed using the ISim simulator, depicted in the subsequent figure.

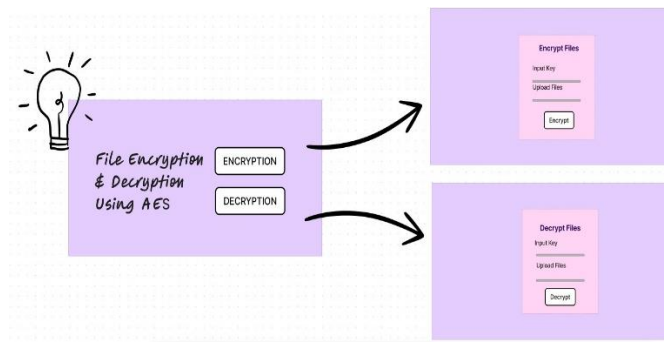


Likewise, for the decryption mechanism, a specialized test bench was developed to evaluate the congruence of the decrypted plaintext with predetermined reference values, incorporating both the ciphertext and the corresponding key. The illustration provided in the following figure validates the decryption process, demonstrating its compliance with the verification standards and affirming its operational reliability and precision within our cryptographic framework.



The software component of our system is structured around a streamlined, user-friendly website comprising three distinct pages. The initial page presents users with a clear choice between encryption and decryption functionalities. Following this selection, users are directed to either the encryption or decryption page, depending on their choice, facilitating a focused and efficient user experience

without unnecessary complexity or additional headings.



File Encryption & Decryption Using AES

Encrypt

Decrypt

Encrypt Files

Input Key

128

Upload Document

Choose File sample.txt

Encrypt

Decrypt Files

Input Key

128

Upload Document

Choose File sample.txt

Decrypt

Plain text of the sample file

sample - Notepad

File Edit Format View Help

hello world!

Encrypted text file after encryption.

sample - Notepad

File Edit Format View Help

ugKfkzeq12WvjKS2Y+J1xZ5c1Q0=

Decrypted text file after encryption.

sample - Notepad

File Edit Format View Help

hello world!

VI. CONCLUSION

In conclusion, the development of the RISC-V Scalar Crypto Extension represents a pivotal advancement in cryptographic capabilities specifically designed for 32-bit RISC-V processors. Through the strategic utilization of the

ESP32C3 microcontroller, our project has achieved notable success in implementing AES encryption and decryption with a 128-bit key, demonstrating the viability of efficient, hardware-based cryptographic solutions. This endeavor has not only enhanced the security features of the RISC-V architecture but also provided invaluable insights into optimization techniques and the importance of user-centric interfaces for secure data processing. Despite the progress made, we acknowledge the existence of unresolved challenges, including scalability, interoperability, and the need for continuous adaptation to evolving cryptographic standards. These issues underscore the complexity of deploying the RISC-V Scalar Crypto Extension in practical settings and highlight areas requiring further investigation and development.

As we look to the future, the groundwork laid by this project opens new avenues for research and innovation. Addressing the remaining challenges, exploring a broader array of cryptographic algorithms, and ensuring the system's adaptability to future standards are imperative steps toward realizing the full potential of hardware-based cryptography within the RISC-V ecosystem. Our work contributes to the ongoing dialogue in the field, providing a foundation from which to enhance data security in an increasingly digital world.

By tackling these challenges head-on and seizing the opportunities for further research, we can continue to advance the state of cryptographic security in RISC-V architectures. This research not only bridges existing gaps but also sets the stage for future innovations, ensuring that the RISC-V architecture remains at the forefront of secure and efficient data processing technologies.

REFERENCES

- [1] Stoffelen, K. (2019, September). Efficient cryptography on the RISC-V architecture. In International Conference on Cryptology and Information Security in Latin America (pp. 323-340). Cham: Springer International Publishing.
- [2] Marshall, B., Newell, G. R., Page, D., Saarinen, M. J. O., & Wolf, C. (2020). The design of scalar AES Instruction Set Extensions for RISC-V. Cryptology ePrint Archive.
- [3] Nişancı, G., Flikkema, P. G., & Yalçın, T. (2022). Symmetric Cryptography on RISC-V: Performance Evaluation of Standardized Algorithms. Cryptography, 6(3), 41.
- [4] Wang, W., Han, J., Cheng, X., & Zeng, X. (2021). An energyefficient crypto-extension design for RISC-V. Microelectronics Journal, 115, 105165.
- [5] Vales, V. B., Fernández, O. C., Domínguez-Bolaño, T., Escudero, C. J., & García-Naya, J. A. (2022). Fine time measurement for the Internet of things: a practical approach using ESP32. IEEE Internet of Things Journal, 9(19), 18305-18318.
- [6] Cameron, N. (2023). ESP32 Microcontroller. In ESP32 Protocols and Communication: Application of Communication Protocols with ESP32 Microcontroller (pp. 1-54). Berkeley, CA: Apress.
- [7] Radhi, B. M., & Hussain, M. A. (2023). Smart Building Security using ESP32 based AES One Bio-key and Owner's Biometrics Encryption Technology. J. Basrah Res.(Sci.), 49(2), 30-47.
- [8] Maier, A., Sharp, A., & Vagapov, Y. (2017, September). Comparative analysis and practical implementation of the ESP32 microcontroller module for the internet of things. In 2017 Internet Technologies and Applications (ITA) (pp. 143-148). IEEE.
- [9] Daemen, J., & Rijmen, V. (1999). AES proposal: Rijndael.
- [10] Waterman, A., Lee, Y., Patterson, D., Asanovic, K., level Isa, V. I. U., Waterman, A., ... & Patterson, D. (2014). The RISC-V instruction set manual. Volume I: User-Level ISA, version, 2.
- [11] Lu, T. (2021). A survey on risc-v security: Hardware and architecture. arXiv preprint arXiv:2107.04175.

- [12] Asanović, K., & Patterson, D. A. (2014). Instruction sets should be free: The case for risc-v. EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146.
- [13] O'Connell, D., Ramsey-Musolf, M. J., & Wise, M. B. (2007). Minimal extension of the standard model scalar sector. *Physical Review D*, 75(3), 037701.
- [14] Pan, L., Tu, G., Liu, S., Cai, Z., & Xiong, X. (2021). A lightweight AES coprocessor based on RISC-V custom instructions. *Security and Communication Networks*, 2021, 1-13.
- [15] RISC-V Foundation. The RISC-V Instruction Set Manual, Volume 1: UserLevel ISA, Document Version 2.2, May 2017.

Appendix B

Appendix - Visual Material

Poster

Figure B.1 shows the poster for the project.



Figure B.1: Poster

Standee

Figure B.2 shows the standee for the project.



Figure B.2: Standee

Brochure

Figure B.3 shows the brochure front for the project.



Figure B.3: Brochure-Front

Figure B.4 shows the brochure back for the project.

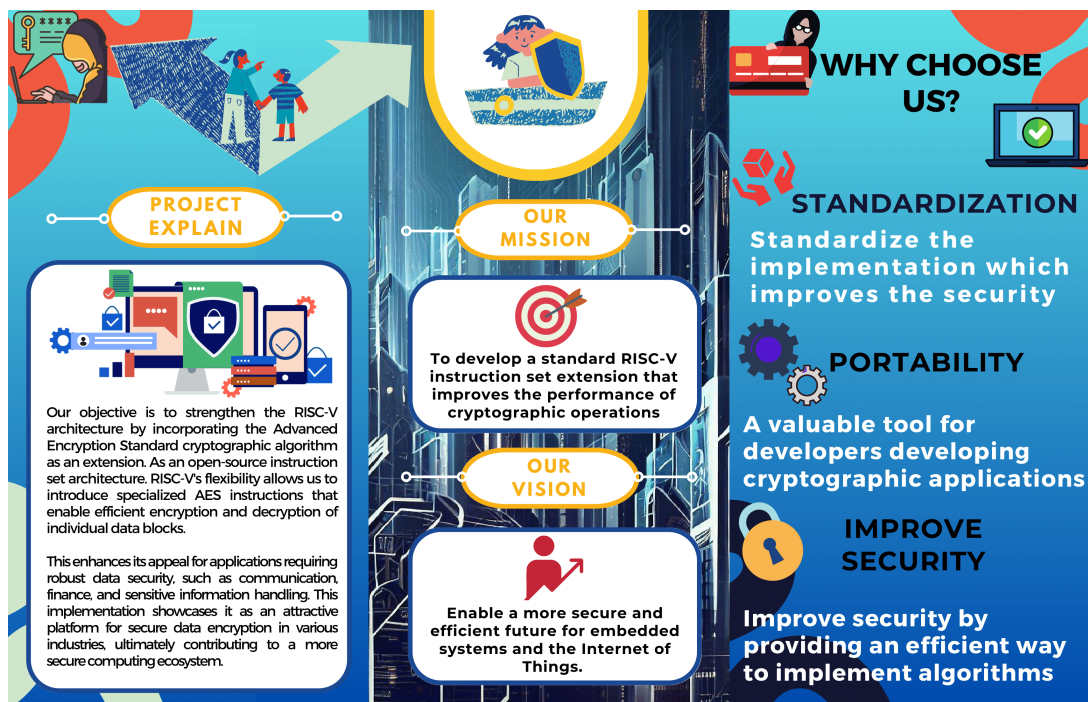


Figure B.4: Brochure-Back

Appendix C

Appendix - Software Requirement Specification

A software requirements specification (SRS) is a document that describes what the software will do and how it will be expected to perform. It also describes the functionality the product needs to fulfill the needs of all stakeholders (business, users).

C.1 Introduction

C.1.1 Product Purpose

Our FYP, "RISC-V Scalar Crypto Extension," aims to fortify the RISC-V architecture with advanced cryptographic functionalities, utilizing the ESP32C3 microcontroller for hardware execution. Users interface with the hardware through a screen and keyboard, enabling real-time encryption and decryption using AES with a 128-bit key. On the software front, a user-friendly website facilitates the encryption of plain text files and the decryption of cipher text files, employing the same AES standard. The integration of the GNU Toolchain compiler ensures a seamless flow of RISC-V code from the software to the ESP32C3 hardware, providing a holistic solution for secure data processing.

C.1.2 Product Scope

The project scope encompasses comprehensive planning and execution, addressing both hardware and software aspects. Key deliverables include the development and integration of the AES-128 bit Algorithm into API, a user-friendly website, and a seamless software-to-hardware flow. Constraints include adherence to cryptographic standards and compatibility testing, while assumptions consider the availability of necessary development tools and the feasibility of ESP32C3 integration. This project scope provides a detailed overview of the envisioned outcome, guiding the project team and stakeholders throughout the development process.

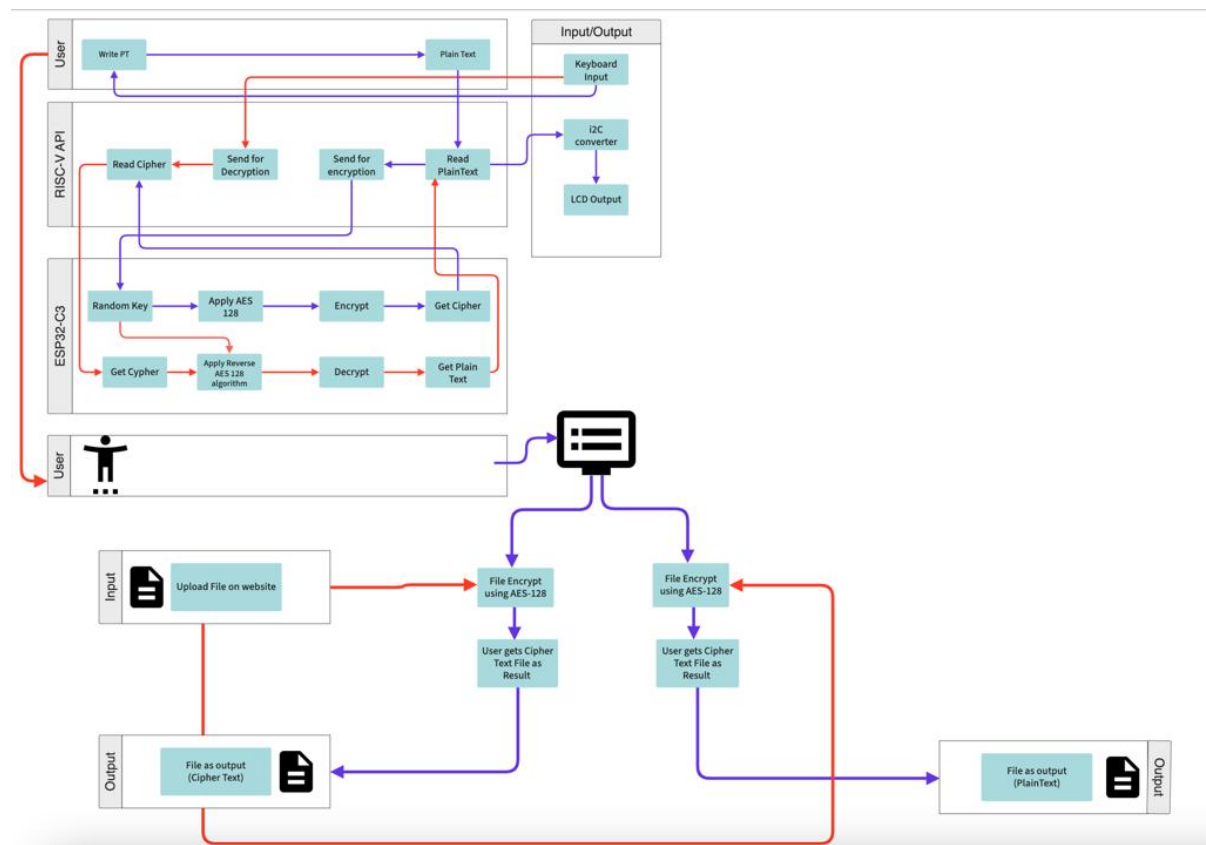


Figure C.1: Line Box Diagram of Hardware and Software System

C.1.3 System Diagram & Description

The project leverages the ESP32C3 microcontroller for efficient hardware execution. Users will interact with the system using a LCD module and keyboard, enabling real-time AES encryption and decryption with a 128-bit key. The software component

features a user-friendly website designed for the seamless encryption of plain text files and decryption of cipher text files, utilizing the same AES standard. A key aspect of the project is the integration of the GNU Toolchain compiler, ensuring a smooth flow of RISC-V code from the software to the ESP32C3 hardware. This integration provides a holistic and secure solution for processing sensitive data.

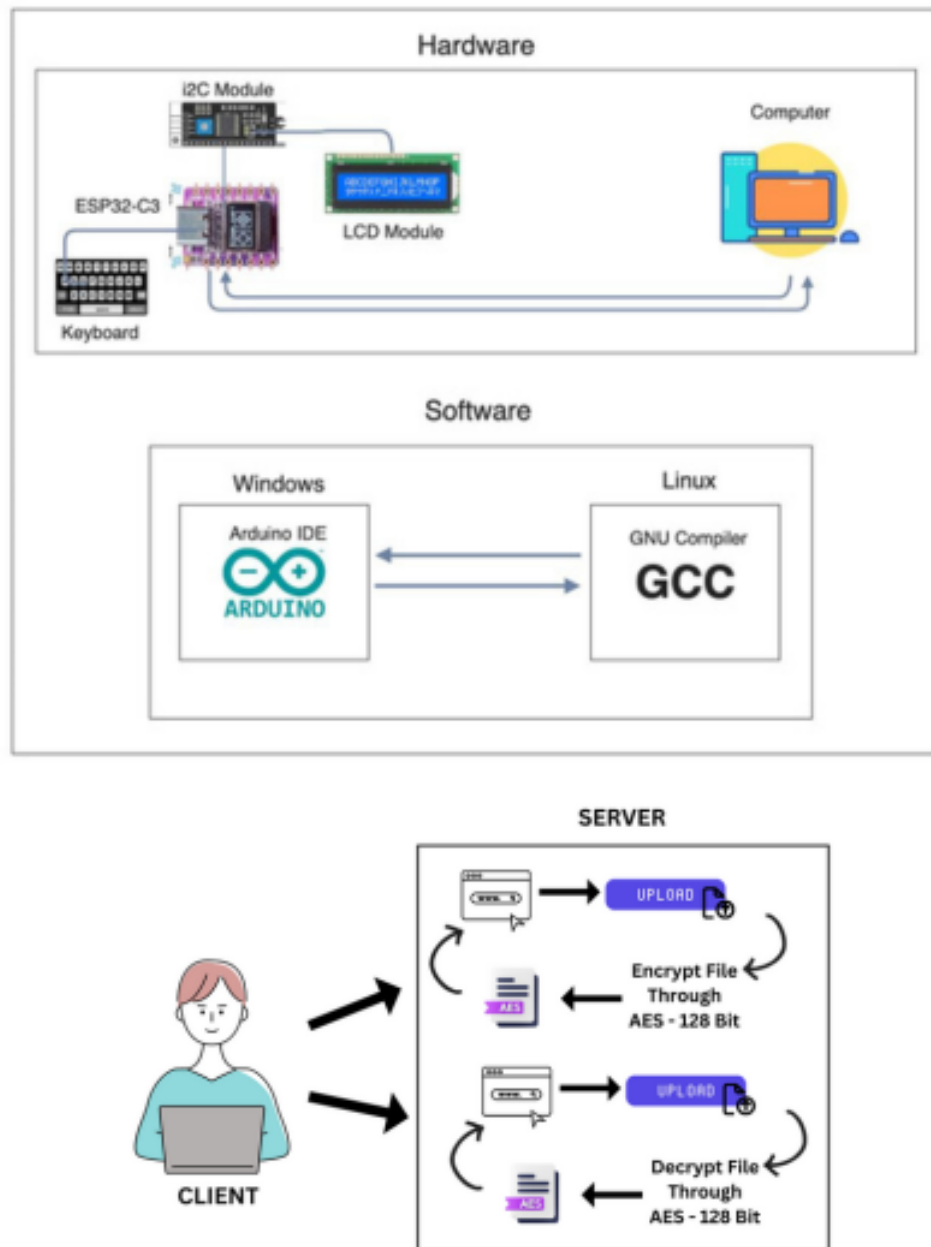


Figure C.2: System Diagram of Hardware and Software

C.2 Project Functional Requirements

1. Product Functional Features

- "RISC-V Scalar Crypto Extension" emerges within the context of the RISC-V architecture, an open-source instruction set architecture that has gained prominence for its flexibility and adaptability. Originating from the need for enhanced security measures, our product addresses this demand by integrating advanced cryptographic functionalities into the RISC-V framework. This extension utilizes the ESP32C3 microcontroller as its hardware foundation. As an innovative solution, it leverages the evolving landscape of RISC-V, positioning itself at the forefront of advancements in secure computing.
- It is the latest addition to the growing RISC-V product line. It introduces advanced cryptographic features, enhancing the existing RISC-V architecture by leveraging the ESP32C3 microcontroller for hardware execution. This extension aligns with ongoing advancements, showcasing a commitment to innovation.

2. Operating Environment

- The software operates on the ESP32C3 microcontroller as the hardware platform, supporting a web-based interface for users
- Windows 10 and Ubuntu
- The software operates on the XAMPP platform for the website.

3. Design & Implementation Constraints

- **Memory Constraints:**

Due to the limited RAM of microcontrollers, the code carefully manages memory usage, especially in resource-intensive tasks such as cryptographic operations using libraries like 'lib/aes.php' and 'lib/aesctr.php.'

- **Processing Power:**

The software accounts for the constrained processing power of microcontrollers, ensuring that operations like encryption or decryption do not overly burden the system during execution.

- **Compatibility Constraints:**

Specifically designed for ESP32-C3, the code may not be directly compatible with other microcontroller architectures or alternative ESP32 variants.

- **Dependency Constraints:**

The code depends on external libraries, such as 'lib/aes.php' and 'lib/aesctr.php.' Future updates or changes to these libraries may affect compatibility.

- **Code Portability:**

Limited portability is expected, restricting the code's ease of adaptation to different microcontroller platforms or environments.

- **Development Tool Constraints:**

The code development relies on specific tools, potentially including the Arduino IDE and GNU Toolchain, which may limit the flexibility of development environments.

C.2.1 Project Functional Requirements

1. Cryptographic Operations:

- Requirement: Enable users to perform AES 128-bit encryption and decryption on plain text and cipher text files.
- Rationale: Facilitate secure data processing using the RISC-V architecture and ESP32-C3 microcontroller.

2. User Interface Interaction:

- Requirement: Implement a user-friendly web interface accessible globally for interacting with the hardware, allowing users to input files and commands.
- Rationale: Enhance user experience by providing a seamless platform for encryption and decryption operations.

3. Memory Management:

- Requirement: Optimize memory usage, especially during cryptographic operations, considering the limited RAM of microcontrollers like ESP32-C3.
- Rationale: Ensure efficient utilization of resources to prevent memory-related issues.

4. Processing Efficiency:

- Requirement: Address limited processing power constraints of microcontrollers, ensuring smooth execution of cryptographic tasks without significant impact on system performance.
- Rationale: Enhance overall system efficiency and responsiveness.

5. Compatibility with ESP32-C3:

- Requirement: Develop code specifically tailored for ESP32-C3, acknowledging potential incompatibility with other microcontroller architectures or ESP32 variants.
- Rationale: Ensure the software is optimized for the designated hardware platform.

6. Library Dependencies:

- Requirement: Manage dependencies on external libraries, such as 'lib/aes.php' and 'lib/aesctr.php,' ensuring compatibility and considering potential updates to these libraries.
- Rationale: Address the need for external resources while maintaining stability and adaptability.

7. Portability Considerations:

- Requirement: Acknowledge limited portability, restricting the code's adaptability to different microcontroller platforms or environments.
- Rationale: Recognize the project's specific focus on ESP32-C3 and potential challenges in porting to other platforms.

8. Development Tool Usage:

- Requirement: Utilize specific development tools, including the Arduino IDE and GNU Toolchain, for coding and compilation.
- Rationale: Ensure consistency and compatibility during the development process.

9. Security Requirements::

- Key Management: The code handle key management securely, including key generation and secure key storage.
- Cryptographic Security: Adhered to best practices for cryptographic security, ensuring that the AES algorithm is implemented correctly and securely.

C.2.3 UX Prototype

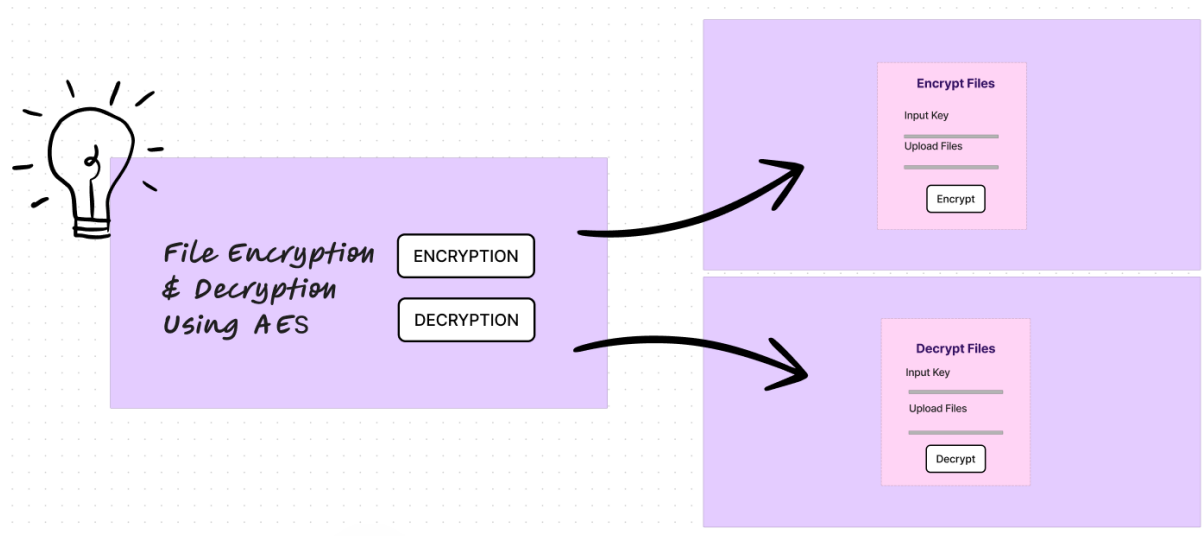


Figure C.5: UX Prototype

C.3 Project Non-Functional Requirements

C.3.1 Performance Evaluation

1. Cryptographic Operations:

- Functional Requirement: Enable users to perform AES 128-bit encryption and decryption on files upto 10MB.
- Performance Goal: Complete encryption and decryption tasks within a maximum processing time of 5 seconds.
- Rationale: Timely execution of cryptographic operations to meet user expectations and ensure efficient data processing.

2. User Interface Responsiveness:

- Functional Requirement: Implement a user-friendly web interface for global users to interact with the system.
- Performance Goal: Maintain an average page load time of 3 seconds for seamless user experience.

- Rationale: Enhance system usability by providing a responsive interface, minimizing delays during user interactions.

3. Memory Management:

- Functional Requirement: Optimize memory usage during cryptographic operations on the ESP32-C3 microcontroller.
- Rationale: Efficient memory management to maintain system reliability and prevent performance degradation.
- Performance Goal: Ensure cryptographic tasks do not exceed 80

C.3.2 Safety Requirements

1. Data Integrity:

- Regularly backup critical data and provide a warning mechanism for potential data integrity risks.

2. Error Handling:

- Implement error codes and clear error messages to guide users in resolving issues without compromising system integrity.

3. Compliance with Cryptographic Standards:

- Regularly update the cryptographic algorithms in accordance with industry best practices and standards.

4. User Data Protection:

- Implement secure authentication mechanisms and data encryption to protect user information. Regulatory Compliance:

C.3.3 Security Requirements

- **Data Protection:**

Security Requirement: Implement robust measures to protect data used or created by the product, ensuring confidentiality and integrity. Privacy Measure: Employ encryption algorithms, such as AES 128-bit, for secure handling of sensitive information.

C.3.4 Business Perspective/Requirements

1. Target Industry/ Market

The "RISC-V Scalar Crypto Extension" targets the embedded systems and microcontroller industry. This includes companies and developers involved in the development of secure embedded systems, particularly those utilizing the RISC-V architecture. The market focus is on products and services related to cryptographic extensions, emphasizing security.

2. Competitors Analysis

Key competitors in the industry include companies developing cryptographic extensions for microcontrollers, with a focus on enhancing data security within embedded systems. Potential competitors may include firms specializing in hardware-based encryption solutions and those leveraging the RISC-V architecture for secure data processing.

3. Potential Customers

Potential customers include semiconductor manufacturers, embedded system developers, and technology enthusiasts seeking advanced cryptographic capabilities for their projects. These individuals or organizations have an interest in securing data within microcontroller-based systems and could benefit from the added functionalities offered by the "RISC-V Scalar Crypto Extension."

4. Scalability/ Extensibility

The system is designed with scalability and extensibility in mind to accommodate the evolving business needs of semiconductor manufacturers and embedded system developers. As the demand for secure data processing grows, the "RISC-V Scalar Crypto Extension" can easily adapt and scale to meet the increased requirements, ensuring flexibility and future-proofing for businesses in the target industry.

C.4 Software Design Methodology

C.4.1 Design Models

The applicable models may include:

- **Use Case Diagram:**

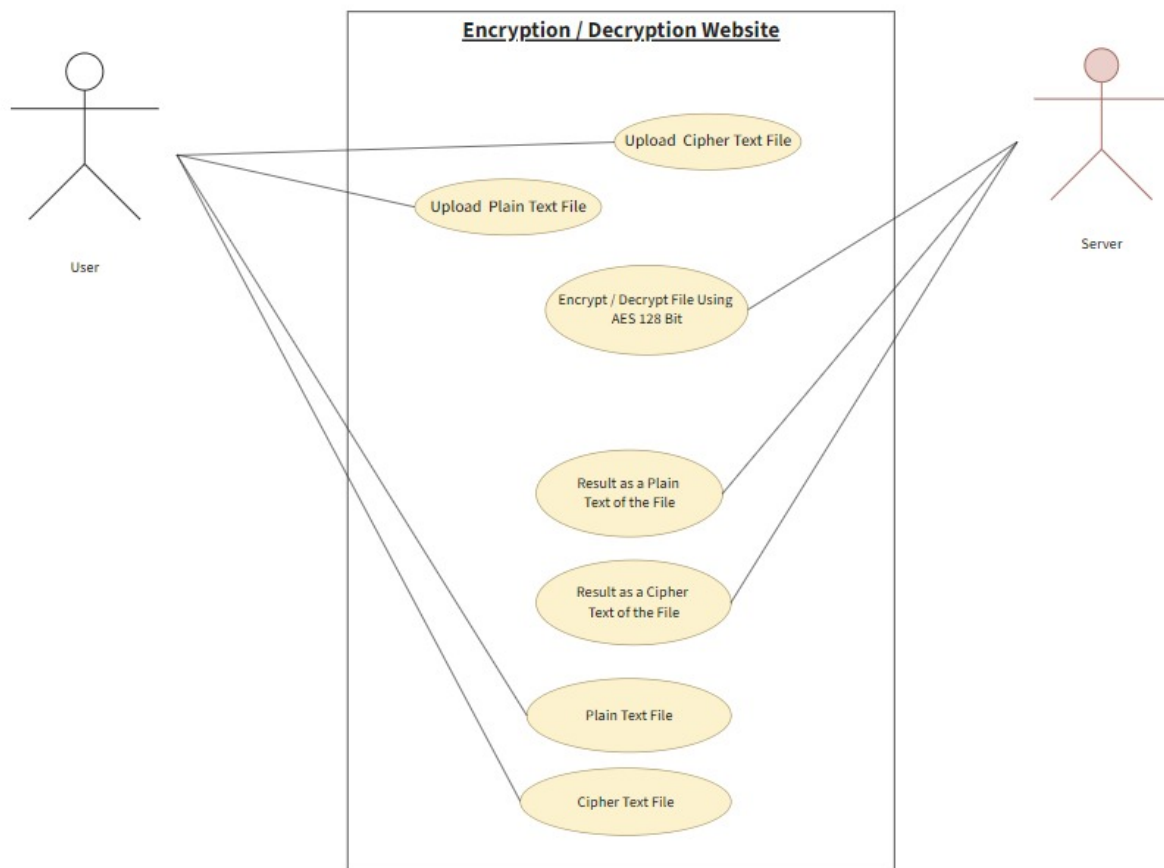


Figure C.6: Use Case Of Encryption/Decryption Website

In this encryption/decryption website, the user can upload a plain text file. The server will then encrypt it using the AES 128-bit algorithm and generate the corresponding cipher text file. The user can later decrypt this cipher text file using the AES 128-bit algorithm on the server, resulting in the regeneration of their original plain text file.

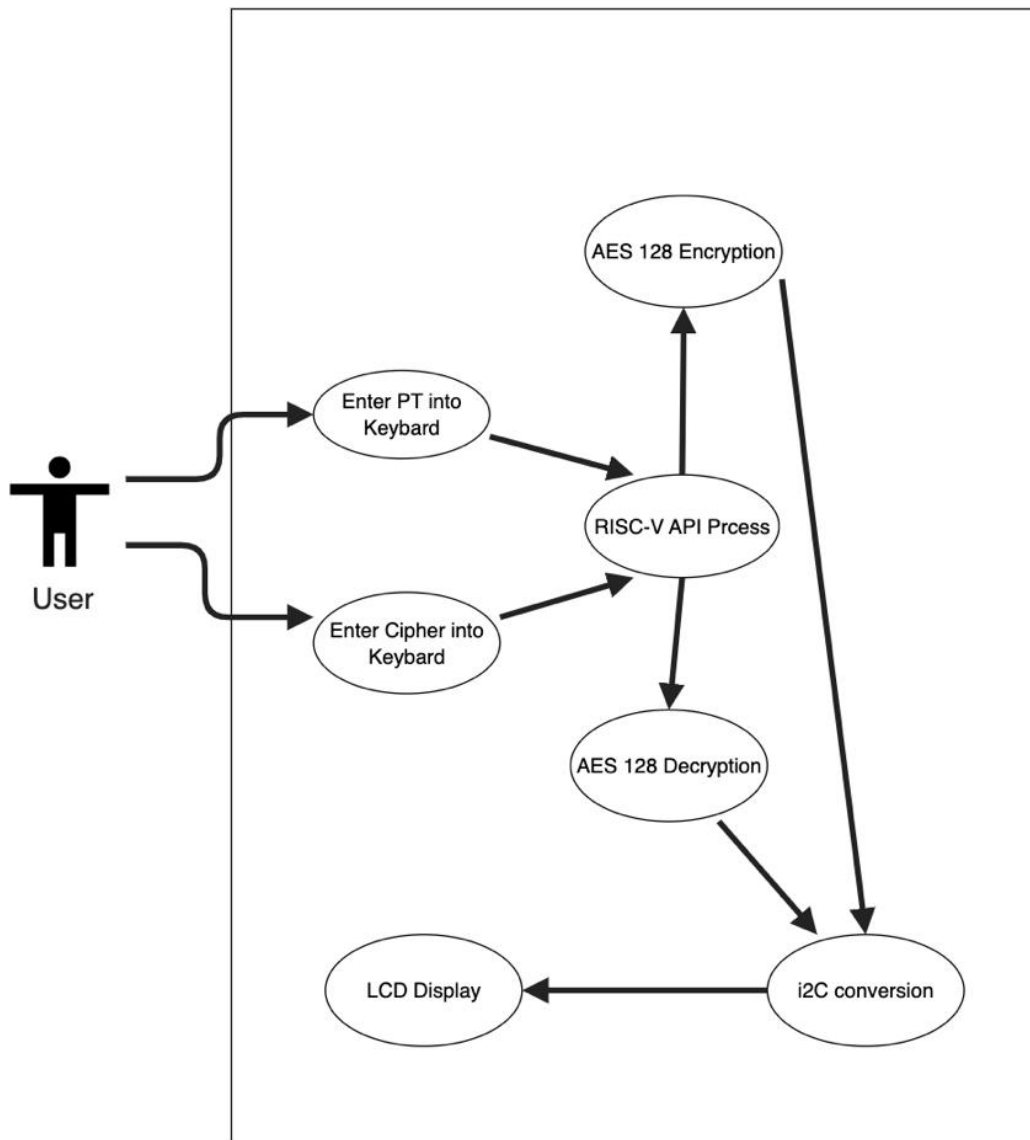


Figure C.7: Use case of hardware

The user initiates the process by inputting plaintext through the keyboard. The RISC-V API process then takes charge of processing the user's input, preparing it for both encryption and decryption operations. The processed data is then

handed over to the AES-128 encryption module, which performs the encryption operation. Subsequently, the encrypted data undergoes I2C conversion, adapting it for communication via the I2C protocol. Simultaneously, the decrypted data follows a similar path, going through the AES-128 decryption process and I2C conversion. Finally, both the encrypted and decrypted data are transmitted to the LCD module, where they are displayed, providing real-time feedback to the user.

- **Sequence Diagram:**

UML Sequence Diagram

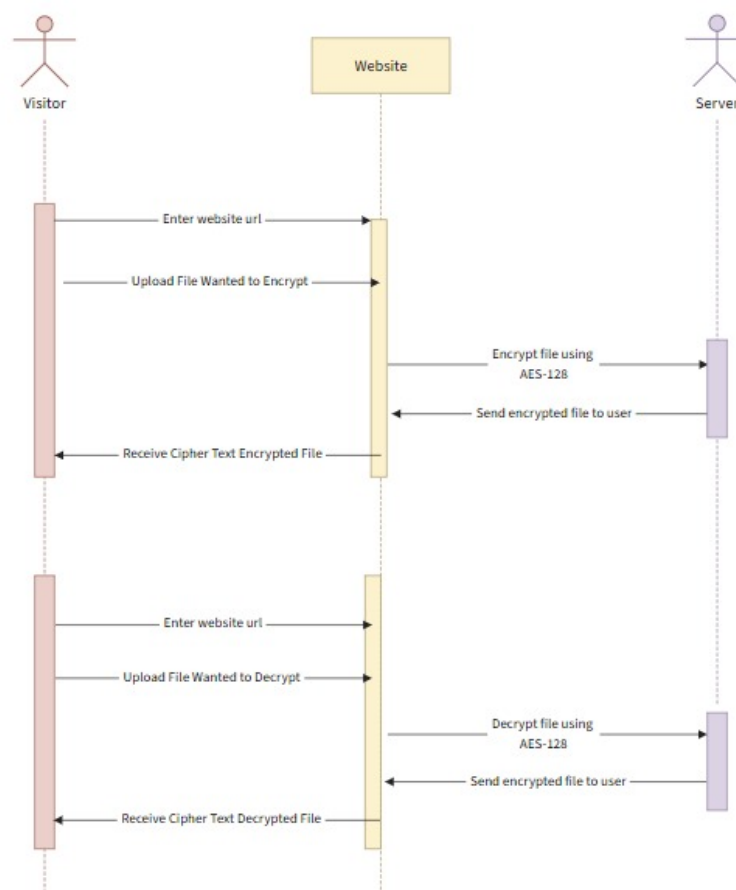


Figure C.8: Sequence Diagram of Encryption And Decryption

In this sequence diagram, the encryption process, a visitor uploads a file on the website, triggering the server to encrypt it using AES-128. The encrypted file is then sent back to the website and provided to the visitor. Similarly, in decryption, the visitor uploads an encrypted file, prompting the server to decrypt it.

The decrypted file is returned to the website and made available to the visitor, ensuring a secure and streamlined user experience.

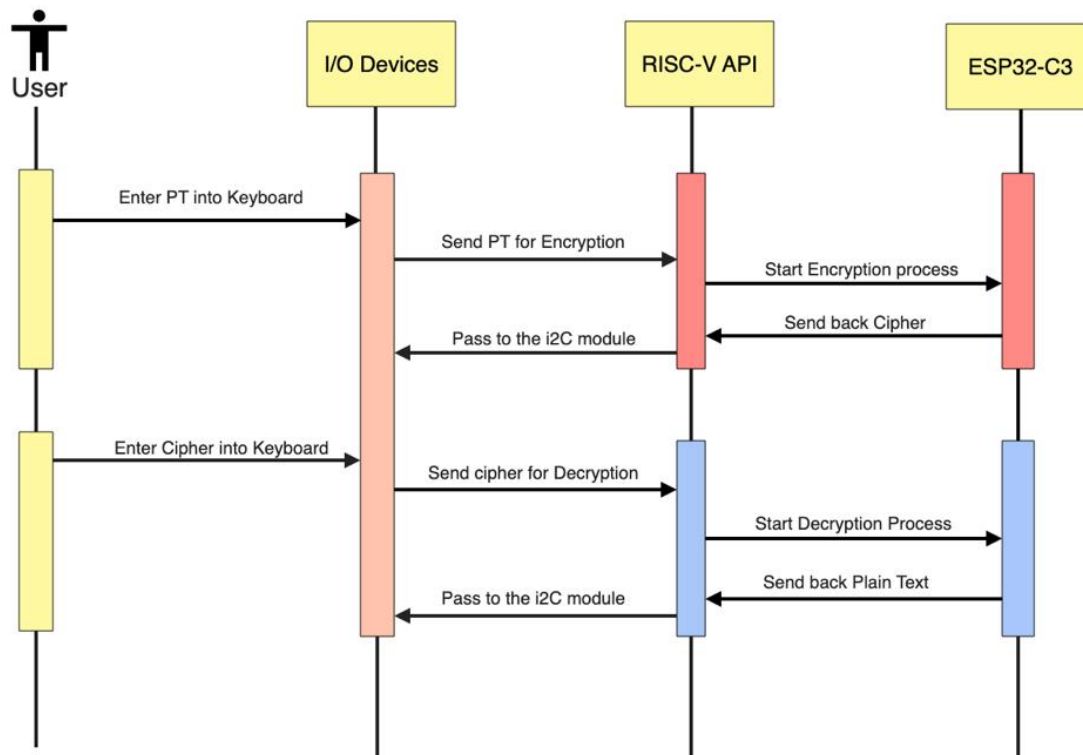


Figure C.9: Sequence Diagram of Hardware

In encryption, the user inputs plaintext via keyboard I/O devices, initiating encryption through the RISC-V API and ESP32-C3. The resulting ciphertext is then processed by the I2C module. In decryption, the user inputs ciphertext, triggering the RISC-V API and ESP32-C3 for decryption. The decrypted plaintext is then passed to the I2C module. This concise process ensures secure and efficient handling of both encryption and decryption operations.

- **Schematice Diagram:**

The schematic diagram depicts the hardware connectivity in the system. The user's keyboard I/O devices are linked to the ESP32-C3 microcontroller, establishing the initial point of interaction. Subsequently, the ESP32-C3 communicates with the I2C module, facilitating data transfer. Finally, the I2C module

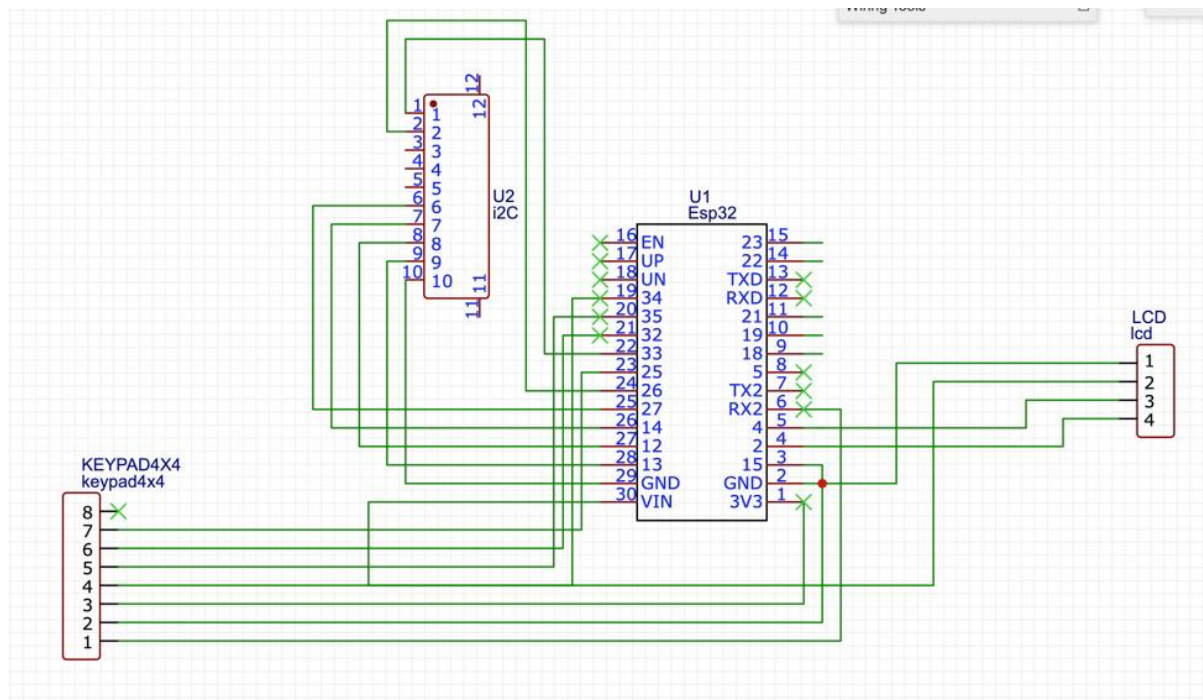


Figure C.10: Schematic Diagram

connects to the LCD module, forming a comprehensive hardware arrangement. This configuration ensures a seamless flow of information from the user's input through the microcontroller and I2C communication to the LCD module, contributing to the overall functionality of the system.

C.4.2 Specialized Algorithms

- In our project we are using Advance Encryption Standard(AES) 128 bit algorithm.

1. Key Expansion

- Generate a set of round keys from the initial secret key using a key expansion algorithm.

– Pseudo Code:

* KeyExpansion(key): // Implementation details for key expansion

2. Initial Round:

- AddRoundKey step using the initial round key.
- **Pseudo Code:**
 - * InitialRound(state, roundKey[0]): // Implementation details for initial round

3. Main Rounds

- A series of rounds involving SubBytes, ShiftRows, MixColumns, and AddRoundKey operations.
- **Pseudo Code:**
 - * MainRounds(state, roundKeys): // Implementation details for SubBytes, ShiftRows, MixColumns, AddRoundKey

4. Final Round:

- A modified set of operations for the final round without MixColumns.
- **Pseudo Code:**
 - * FinalRound(state, roundKeys[10]): // Implementation details for the final round

5. SubBytes, ShiftRows, MixColumns, AddRoundKey:

- Detailed steps for each operation in the Main Rounds.
- **Pseudo Code:**
 - * SubBytes(state): // Implementation details for SubBytes
 - * ShiftRows(state): // Implementation details for ShiftRows
 - * MixColumns(state): // Implementation details for MixColumns
 - * AddRoundKey(state, roundKey): // Implementation details for AddRoundKey

C.5 External Interfaces Requirements

1. User Interfaces

- **File Upload:**

Users can easily upload plaintext files for encryption and decryption. The file upload feature includes a standardized button for selecting files.

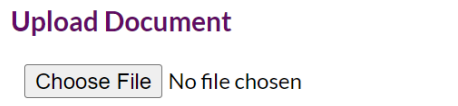


Figure C.11: Upload Button

- **File Encrypt and Decrypt:**

This encryption component includes the interface initiating encryption, and saving the result into a encrypted file and the decryption component features the initiating decryption, and providing the decrypted output.



Figure C.12: Encryption and Decryption button

2. Hardware Interfaces

- **ESP32-C3 Microcontroller:**

- Logical Characteristics: The software operates on the ESP32C3 microcontroller as its primary hardware platform and supports cryptographic operations, memory management, and processing efficiency in compliance with the microcontroller's capabilities.
- physical Characteristics: ESP32-C3 serves as the foundational hardware for executing cryptographic functionalities, enhancing the RISC-V architecture.

- **Operating Systems:**

- Logical Characteristics: Compatible with Windows 10 and Ubuntu operating systems. Adapts its functionality based on the underlying operating system for seamless integration.
- physical Characteristics: Ensures compatibility with the specified operating systems, leveraging their capabilities for a wider user base.

3. Software Interfaces

- **Operating Systems:**

- Windows 10 and ubuntu-22.04.3
- The software product is designed to operate seamlessly on these operating system.

- **Web Server:**

- XAMPP: The web-based interface is hosted on the XAMPP platform, which provides an environment for testing and deploying the website. XAMPP includes components such as Apache for web server capabilities.

- **Development Tools:**

- The development of the software code relies on specific tools, such as GNU Toolchain. This tool facilitate coding, compilation, and integration with the ESP32-C3 microcontroller..

- **External Libraries:**

- 'lib/aes.php' and 'lib/aesctr.php'
- The code has dependencies on external libraries, specifically 'lib/aes.php' and 'lib/aesctr.php.' These libraries contribute to cryptographic operations.

4. Communications Interfaces

(a) Web Interface Communication:

- Requirement: Implement secure communication protocols for interactions through the web interface.
- Standard: Utilize HTTPS to ensure encrypted data transmission and secure user interactions.

(b) Communication Protocols:

- The code itself doesn't directly involve communication interfaces, but if integrated into a larger system, it can conform to relevant communication protocols for data exchange.

(c) Data Formats:

- The library accepts input data in appropriate formats (e.g., byte arrays) and produce output in the desired format.

Bibliography

- [1] V. B. Vales, O. C. Fernández, T. Domínguez-Bolaño, C. J. Escudero, and J. A. García-Naya, "Fine time measurement for the internet of things: a practical approach using esp32," IEEE Internet of Things Journal, vol. 9, no. 19, pp. 18 305–18 318, 2022.
- [2] N. Cameron, "Esp32 microcontroller," in ESP32 Formats and Communication: Application of Communication Protocols with ESP32 Microcontroller. Springer, 2023, pp. 1–54.
- [3] B. M. Radhi and M. A. Hussain, "Smart building security using esp32 based aes one bio-key and owner's biometrics encryption technology," J. Basrah Res.(Sci.), vol. 49, no. 2, pp. 30–47, 2023.
- [4] A. Waterman, Y. Lee, D. Patterson, K. Asanovic, V. I. U. level Isa, A. Waterman, Y. Lee, and D. Patterson, "The risc-v instruction set manual," Volume I: User-Level ISA', version, vol. 2, 2014.
- [5] K. Stoffelen, "Efficient cryptography on the risc-v architecture," in International Conference on Cryptology and Information Security in Latin America. Springer, 2019, pp. 323–340.
- [6] D. O'Connell, M. J. Ramsey-Musolf, and M. B. Wise, "Minimal extension of the standard model scalar sector," Physical Review D, vol. 75, no. 3, p. 037701, 2007.
- [7] K. Asanović and D. A. Patterson, "Instruction sets should be free: The case

- for risc-v,” EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146, 2014.
- [8] B. Marshall, G. R. Newell, D. Page, M.-J. O. Saarinen, and C. Wolf, “The design of scalar aes instruction set extensions for risc-v,” Cryptology ePrint Archive, 2020.
- [9] T. Lu, “A survey on risc-v security: Hardware and architecture,” arXiv preprint arXiv:2107.04175, 2021.
- [10] G. Nişancı, P. G. Flikkema, and T. Yalçın, “Symmetric cryptography on risc-v: Performance evaluation of standardized algorithms,” Cryptography, vol. 6, no. 3, p. 41, 2022.
- [11] L. Pan, G. Tu, S. Liu, Z. Cai, and X. Xiong, “A lightweight aes coprocessor based on risc-v custom instructions,” Security and Communication Networks, vol. 2021, pp. 1–13, 2021.
- [12] W. Wang, J. Han, X. Cheng, and X. Zeng, “An energy-efficient crypto-extension design for risc-v,” Microelectronics Journal, vol. 115, p. 105165, 2021.
- [13] J. Daemen and V. Rijmen, “Aes proposal: Rijndael,” 1999.
- [14] A. Maier, A. Sharp, and Y. Vagapov, “Comparative analysis and practical implementation of the esp32 microcontroller module for the internet of things,” in 2017 Internet Technologies and Applications (ITA). IEEE, 2017, pp. 143–148.